Этот конспект сгенерирован с помощью AI. Система может допускать ошибки в формулах, вычислениях и специфической терминологии. Пожалуйста, относитесь с понимаем и проверяйте конспект!

Основы логики и её интерпретации

Алгебра логики — это раздел математики, в котором высказывания рассматриваются как объекты с двумя возможными значениями: истина (true) или ложь (false). Каждое высказывание обозначается буквой, например A, и его истинность определяется в рамках конкретной аксиоматической системы. Значение высказывания зависит не только от его содержания, но и от языка, на котором оно записано, а также от логической системы, в которой интерпретируется. Например, утверждение «2 + 2 = 4» истинно в стандартной арифметике, тогда как «1 + 1 = 1» — ложно.

Важным аспектом является то, что истинность высказывания может зависеть от аксиоматической базы: утверждение «Бог есть» может быть истинным в религиозной системе и ложным — в атеистической. Это демонстрирует, что разные логические системы могут быть несовместимы между собой, и попытка построить единую абсолютную истину сталкивается с фундаментальными ограничениями.

Такой подход к логике был опровергнут теоремой Гёделя о неполноте: любая достаточно мощная формальная система либо противоречива, либо неполна — в ней существуют утверждения, которые нельзя ни доказать, ни опровергнуть. Это означает, что универсальной логической системы, способной выразить всё истинное, не существует.

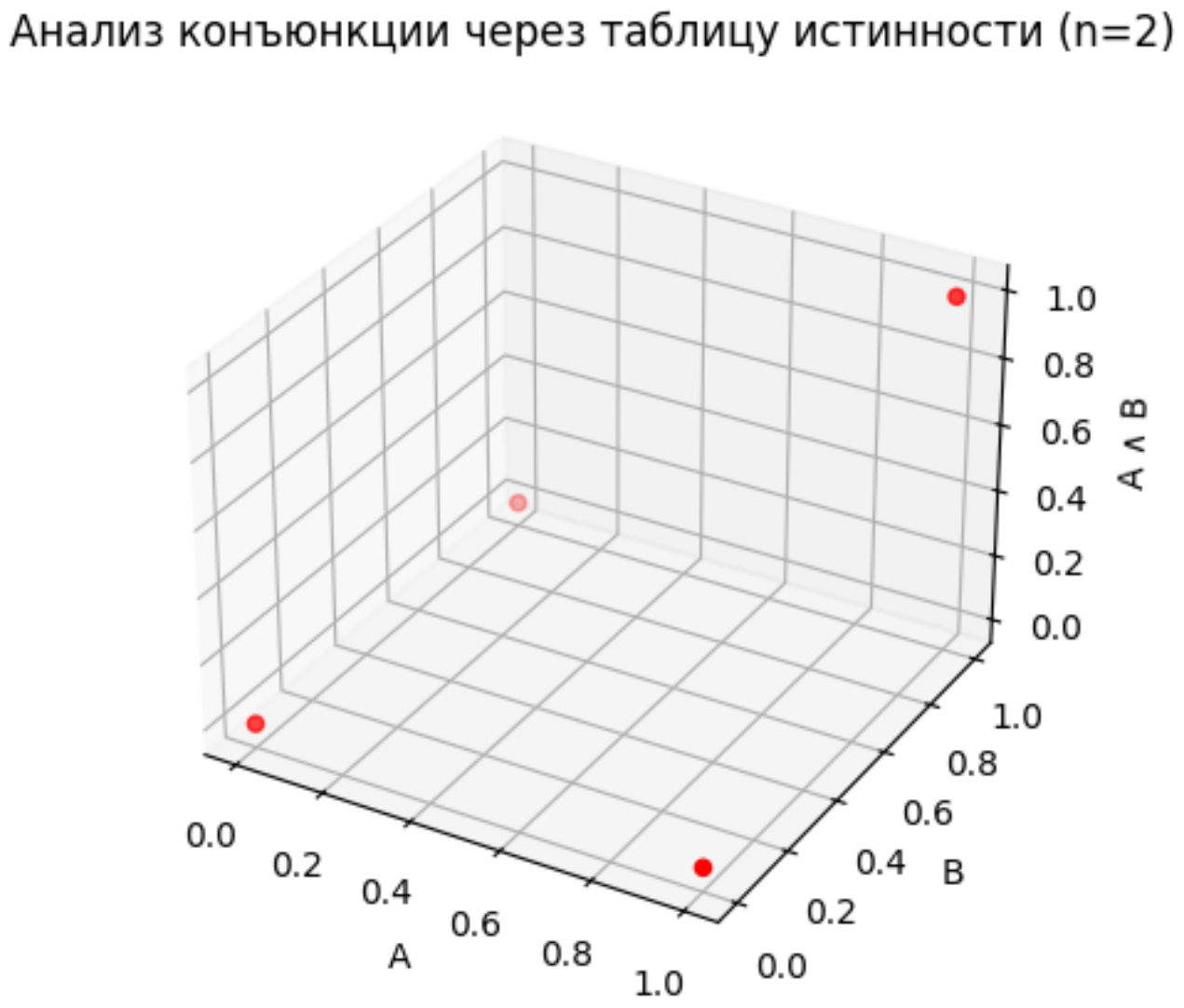
В рамках алгебры логики мы не рассматриваем атомарные высказывания (простые утверждения без внутренних структур), а сосредоточимся на составных — построенных из них с помощью логических операций: конъюнкции ($\wedge$), дизъюнкции ($\vee$), отрицания ($\neg$), импликации ($\rightarrow$) и эквивалентности ($\leftrightarrow$). Эти операции позволяют формализовать рассуждения и строить логические выражения, подлежащие анализу.

Цель — не построение абсолютной истины, а развитие инструмента для строгого анализа высказываний в рамках заданной системы.

Логические операции и их свойства

Алгебра логики оперирует составными высказываниями, построенными из атомарных с помощью логических операций. Простейшей является операция отрицания: она истинна тогда и только тогда, когда исходное высказывание ложно, и наоборот — ложна при истинном исходнике. Следующая по сложности — конъюнкция (логическое «И»), объединяющая два высказывания (атомарные или составные).

Для анализа истинностных значений используется таблица истинности:
- Конъюнкция (A \land B) истинна только если оба операнда истинны

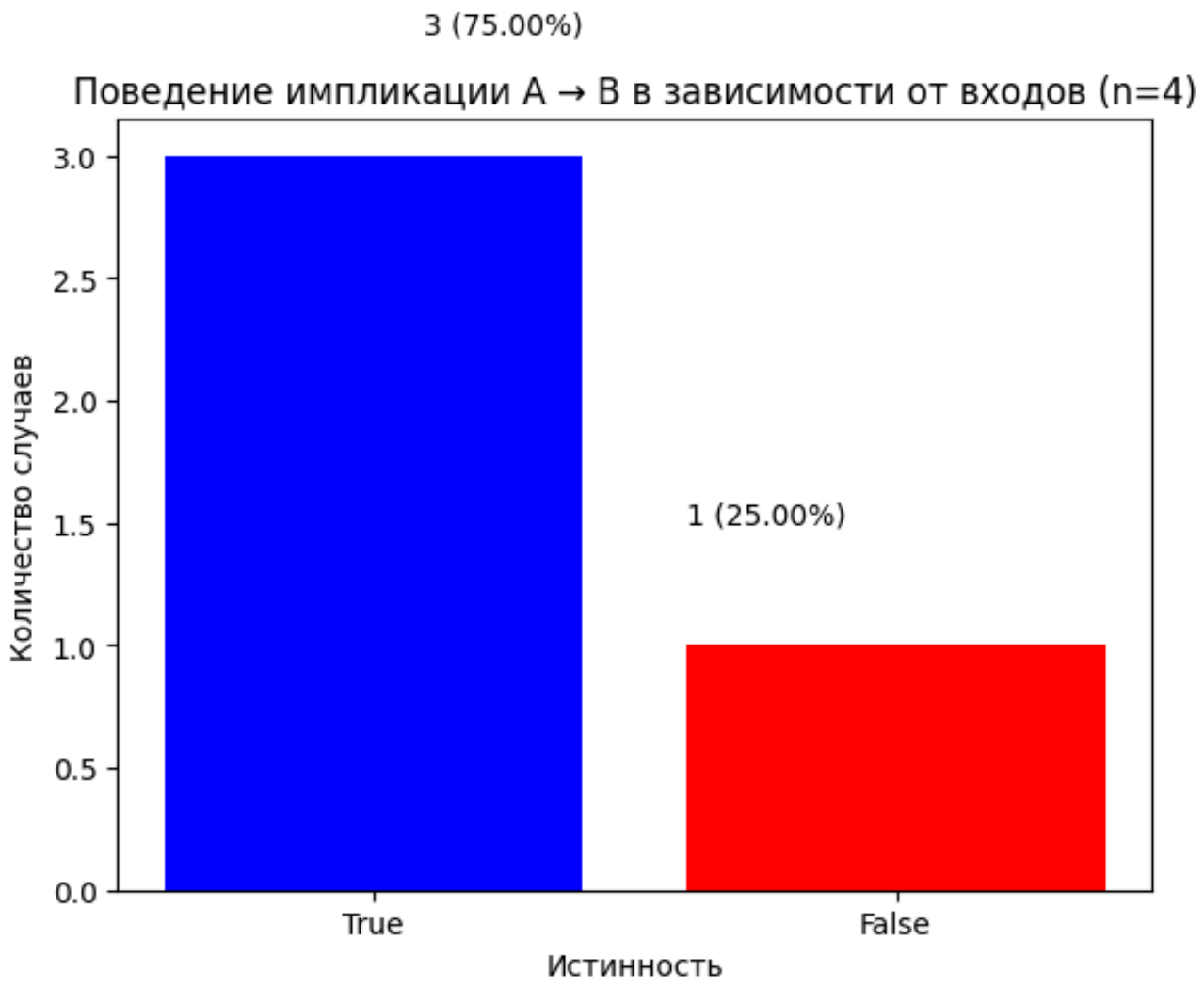


Анализ конъюнкции через таблицу истинности n 2

.
- Дизъюнкция («ИЛИ») — не обсуждалась, но подразумевается как дополнительная

операция.
- Импликация обозначается как (A \rightarrow B), читается «из A следует Б». В логике и математике используется символ (\Rightarrow) при рассуждениях, а в формальных выражениях — (\rightarrow).

Ключевое отличие импликации от эквивалентности:
- Эквивалентность (A \leftrightarrow B) (или (A \equiv B)) истинна только если значения (A) и (B) совпадают.
- Импликация может быть истинной даже при ложном антецеденте, если консеквент истинен



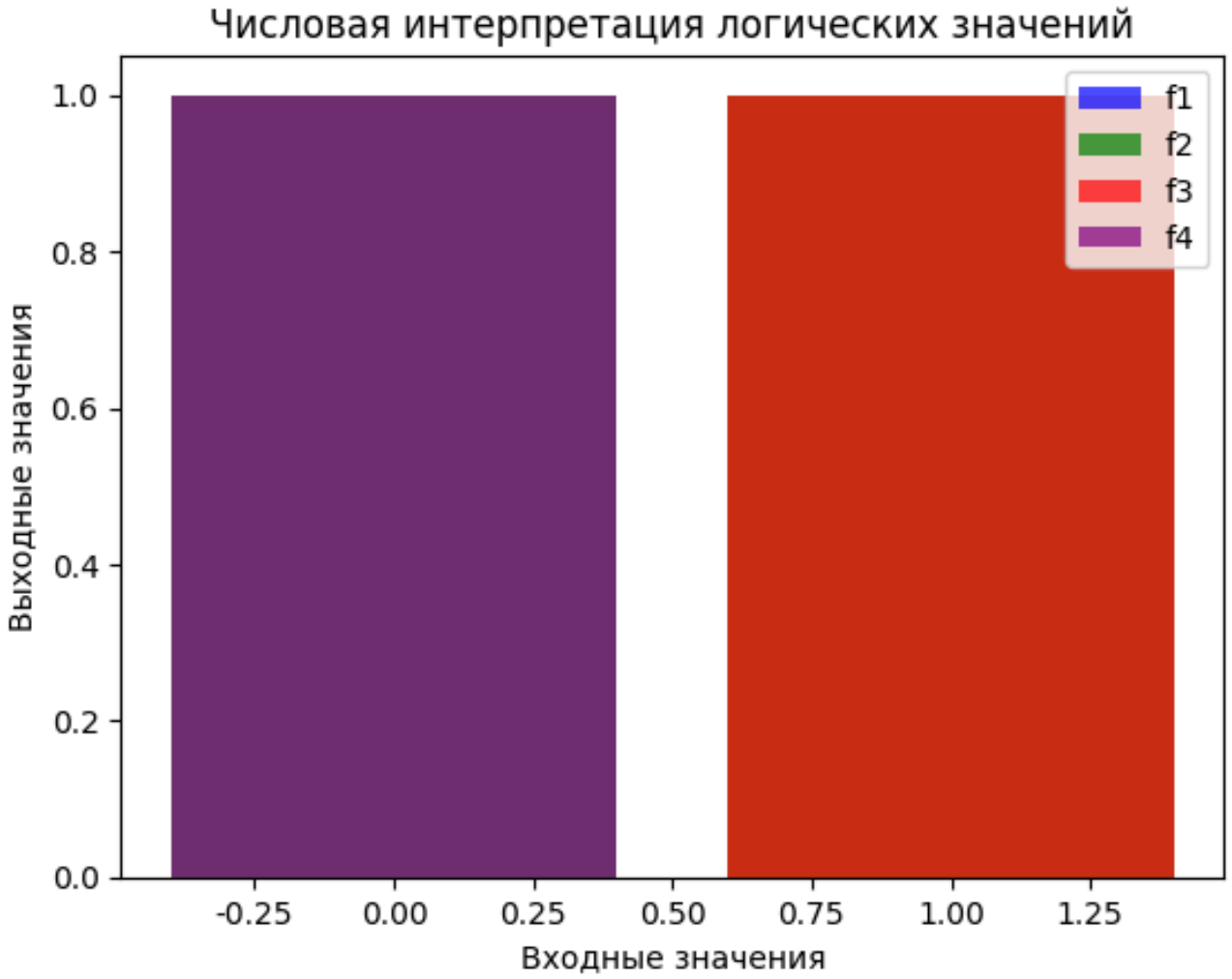
Поведение импликации в зависимости от входов n 4

.

В алгебре логики удобно использовать числовое представление:
- истина = 1,

- ложь = 0.

Это обозначение общепринято в информатике (ЕГЭ), языках программирования и теоретических построениях



Числовая интерпретация логических значений

. Оно позволяет формализовать логические выражения как булевы функции и применять аппарат математической логики к анализу программных конструкций.

Булева алгебра и дискретные функции

Любая логическая функция — это отображение из множества $0, 1$ в себя. При одном аргументе x возможны только два значения: 0 и 1

Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Граф булевых функций одной переменной (4 узла) | MOCK_DATA: {"nodes": [0,1,x,not_x], "edges":

[[0,2],[1,2],[2,3]]} | TASK: Построить граф с 4 узлами — по одному на каждую функцию f(x). Связи между ними показать как возможные композиции (например, f3 → f4 = отрицание результата), используя networkx. Отобразить в виде схемы.]

Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Пространство булевых функций одной переменной | MOCK_DATA: {"inputs": [0, 1], "functions": [{"name": "f1", "values": [0, 0]}, {"name": "f2", "values": [1, 1]}, {"name": "f3", "values": [0, 1]}, {"name": "f4", "values": [1, 0]}]} | TASK: Визуализировать топологию пространства всех возможных функций f(x) при x ∈{0,1}: построить матрицу истинности (2×4), где строки — входы, столбцы — функции. Отобразить как тепловую карту.]

. Соответственно, существует ровно четыре различные функции одной переменной:

- $f_1(x) = 0$ — тождественный ноль;
- $f_2(x) = 1$ — константа единица;
- $f_3(x) = x$ — тождественная функция;
- $f_4(x) = \overline{x}$ — отрицание (логическое «не»), обозначаемое чертой над переменной: $\overline{x}$.

Эти функции исчерпывают все возможные отображения из $0, 1$ в $0, 1$.

Таблицы истинности.
Для двух переменных x и y область определения содержит $2^2 = 4$ комбинации: $(0, 0), (0, 1), (1, 0), (1, 1)$

Структура таблицы истинности для двух переменных

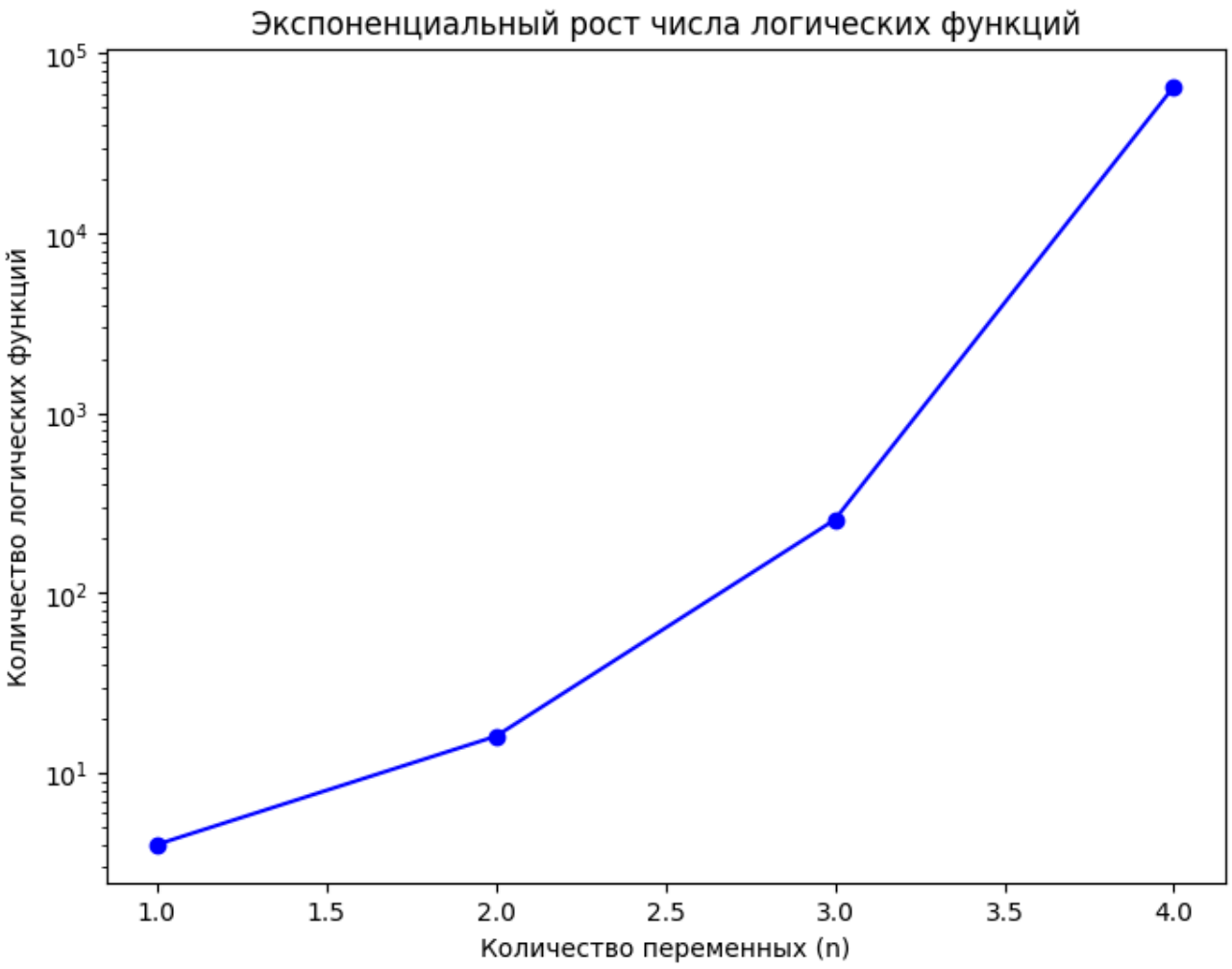
. Каждая функция задаётся значением на каждой из этих точек — выбором 0 или 1.

Количество различных функций двух переменных:

$$2^{2^2} = 2^4 = 16.$$

Для трёх переменных:

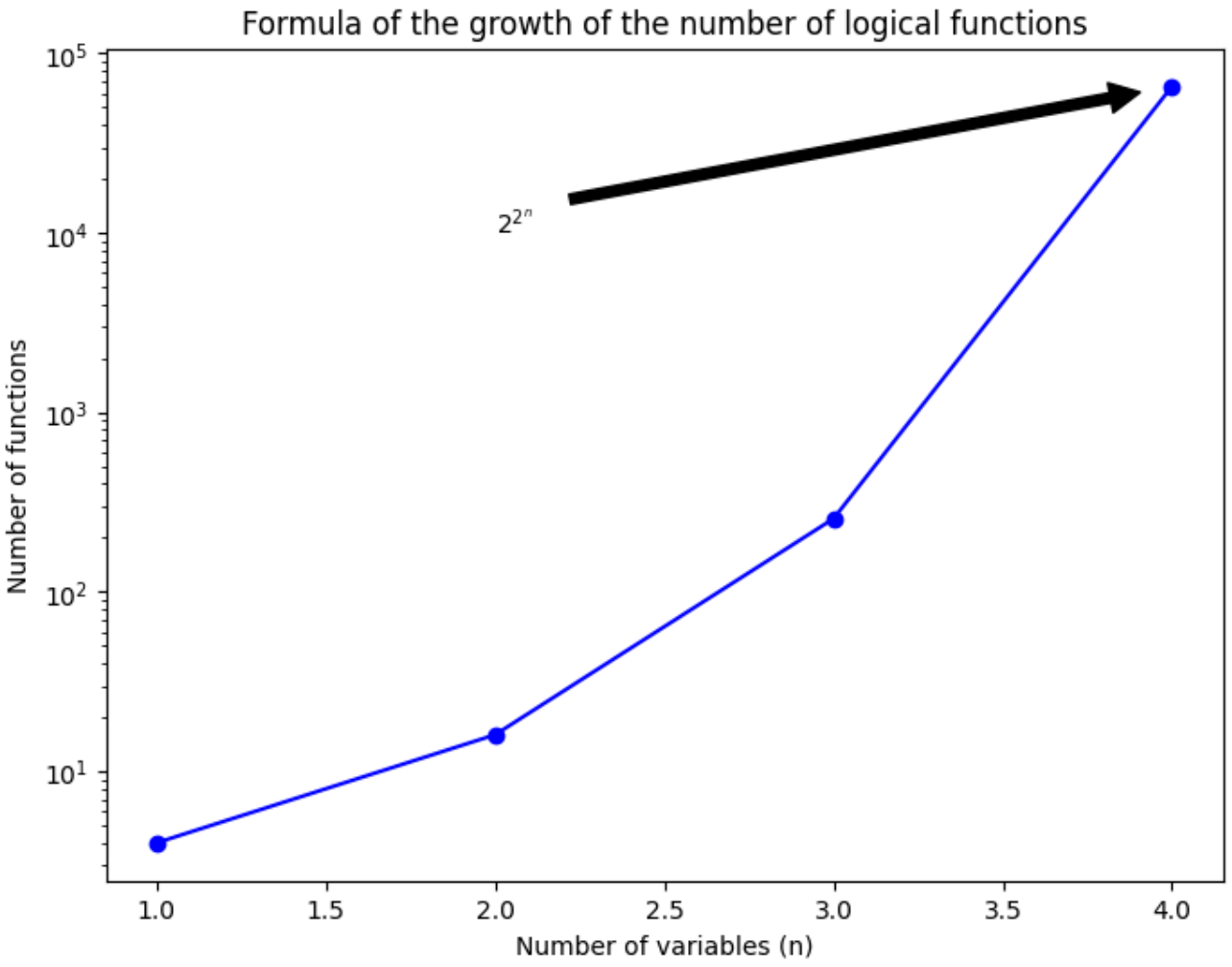
$$2^{2^3} = 2^8 = 256.$$



Экспоненциальный рост числа логических функций

В общем случае количество логических функций n переменных равно:

$$2^{2^n}.$$



Формула роста числа логических функций

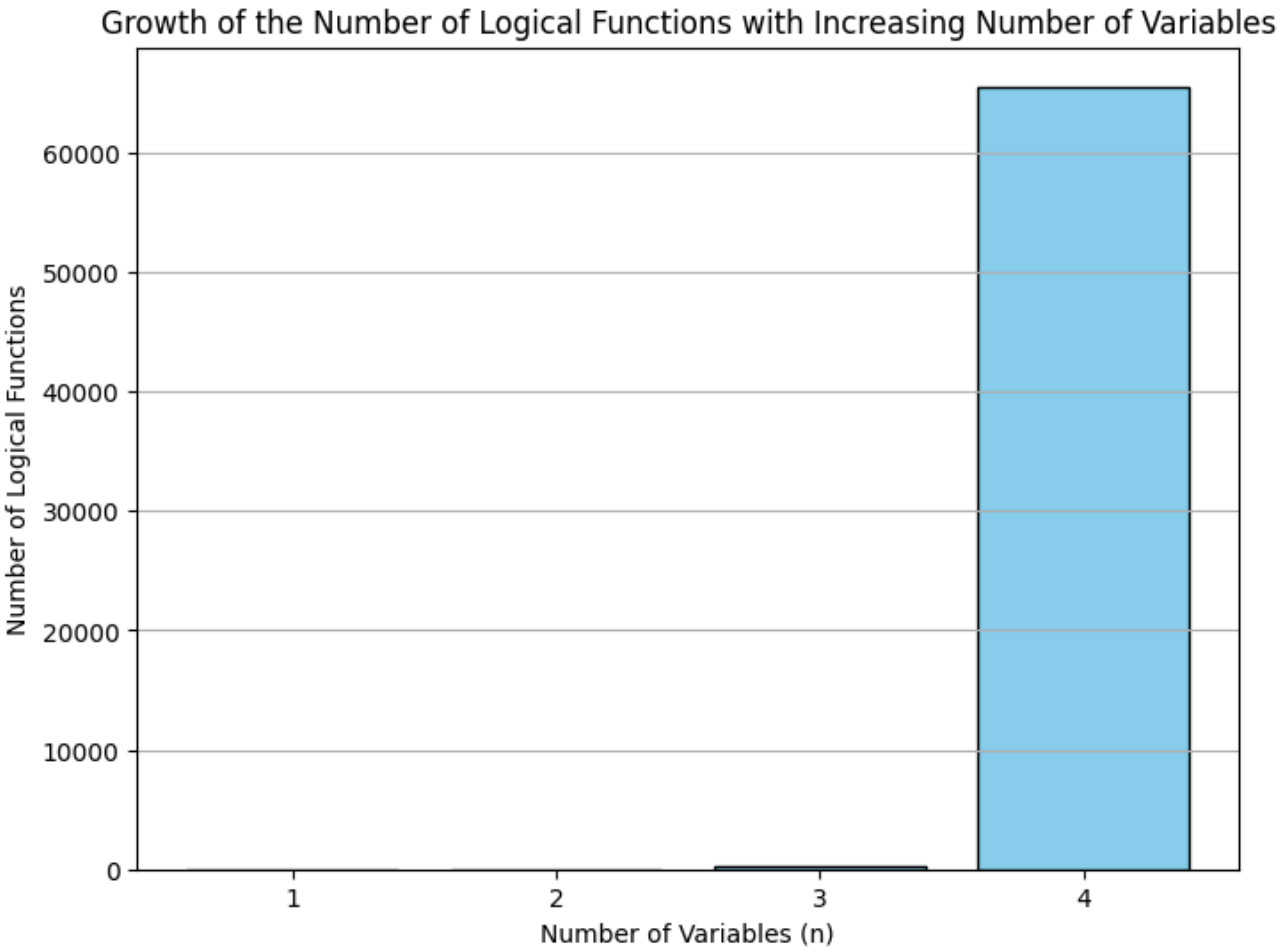
Это демонстрирует экспоненциальный рост числа возможных функций с ростом числа аргументов.

Обозначения операций:

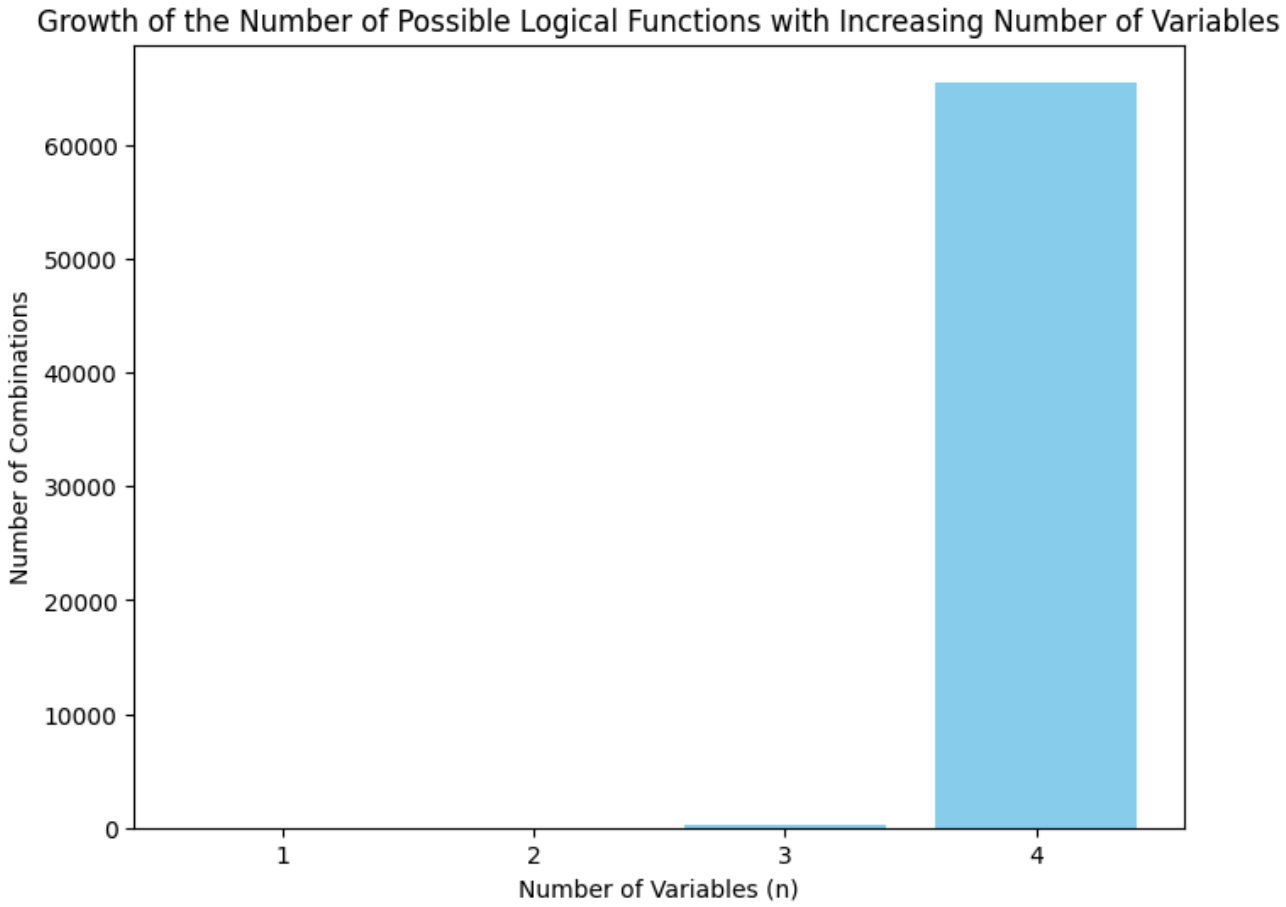
- Отрицание (не): $\overline{x}$ — вместо $\neg x$.
- Логическое И (конъюнкция): $x \wedge y$ — вместо $\cdot$ или $\&$.
- Логическое ИЛИ (дизъюнкция): $x \vee y$ — вместо $+$ или $|$.

Эти обозначения соответствуют стандартной формальной записи в булевой алгебре.

Сколько вариантов я могу комбинации ноликов и единичек написать? 2 тут умножить на 2 тут, правильно? 4 варианта. Или получается, что 2 в степени 2

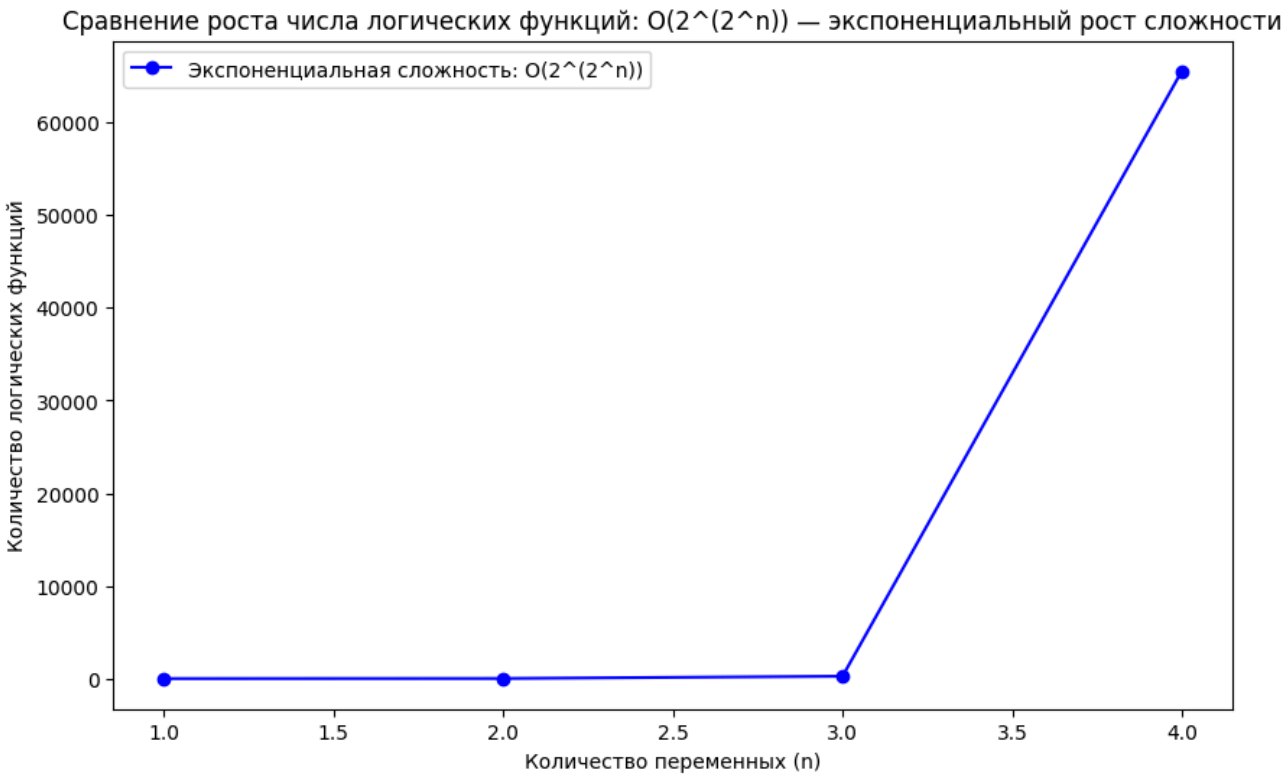


Рост числа логических функций от количества переменных

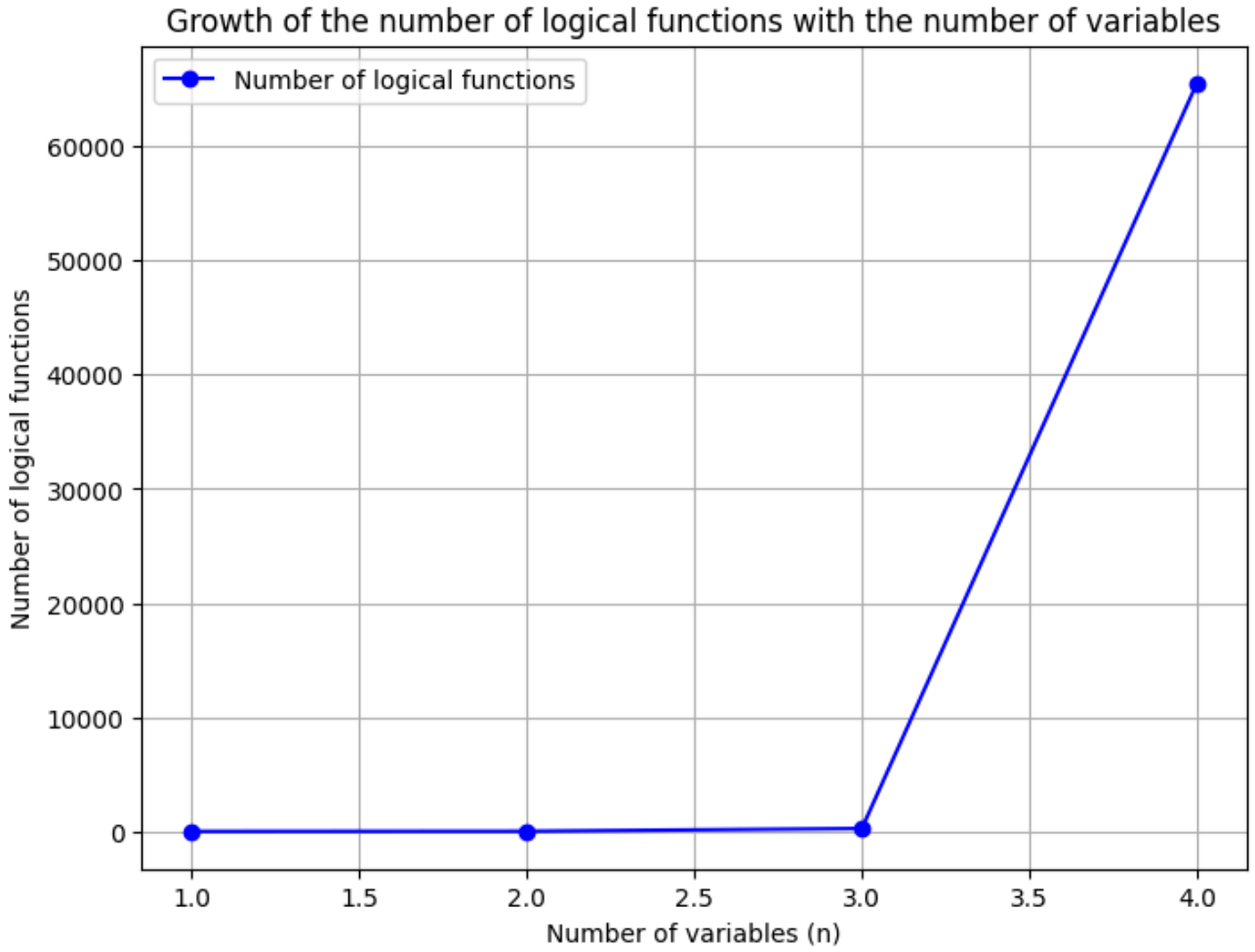


Рост числа комбинаций при увеличении количества переменных

. Если у меня будет 3 логические переменные,
то сколько строк у меня возникнет? Очевидно, 2 в кубе, правильно? То есть у меня здесь 2 в квадрате в квадрате. То есть таких функций у меня сколько? Получается 16 штук



Сравнение роста числа логических функций O 2 2 n



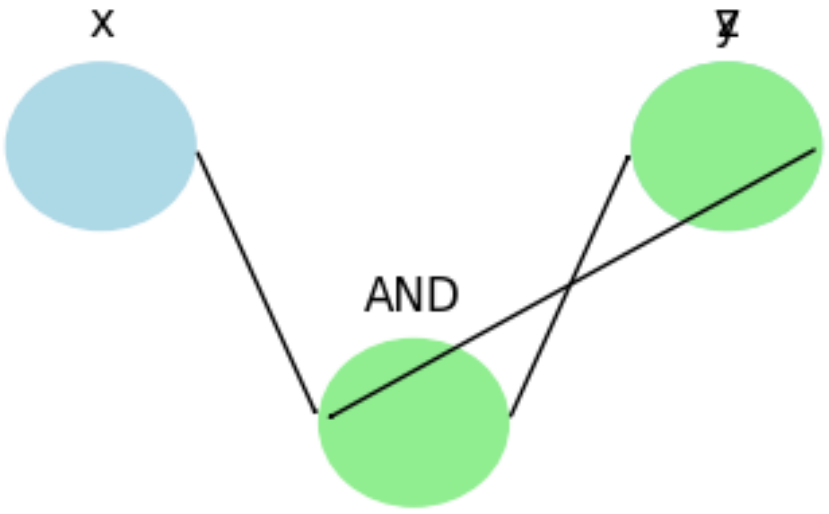
Рост числа логических функций от количества переменных

. 2 в четвертой — 16 штук. 16 штук функций двух переменных. Правильно? Ну, а логических функций трех переменных можно уже поупражняться. Там получится сколько?
$2^{2^3} = 256$ штук. Нам это сейчас уже не очень понадобится,
но там понятно дело, что количество разных функций уже растет. Логические высказывания, задающие эти функции, могут быть достаточно длинными, может быть сложными — правда, мы сейчас пока не обсуждаем, как их записывать.

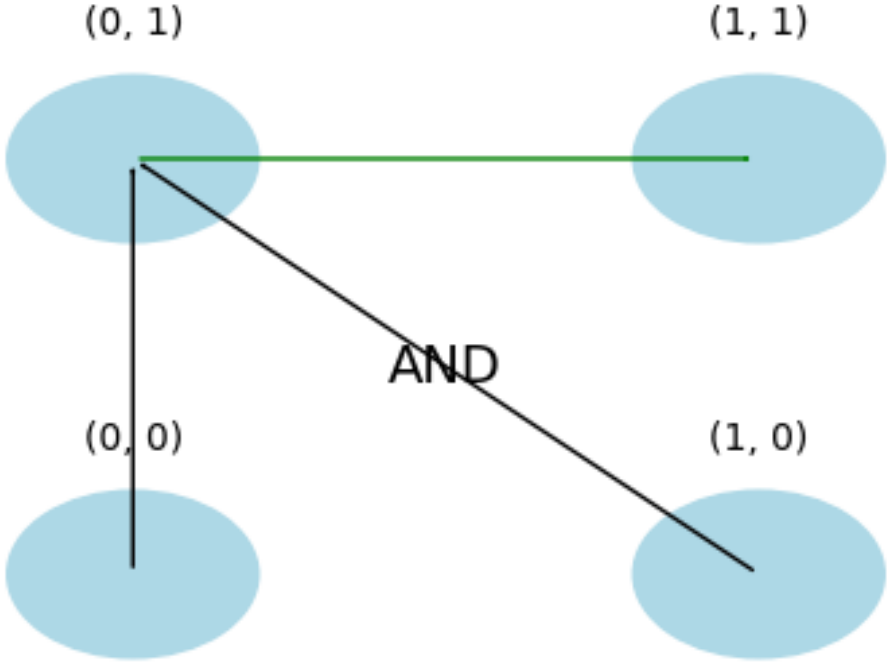
У нас есть кое-какие операции, мы вот кое-какие условно-примитивные назвали. Кстати, между прочим, вот это вот, вот это вот, и вот это вот, и вот это вот, и вот это вот — это что такое? Это логические функции двух переменных. Четыре функции из 16 штук мы узнали. Ну, я вот эти вот 256 сотру — это так упражнение, да? Из них 4 мы уже знаем. Правильно? Так, окей. Значит, из них 4 знаем. И сейчас мы их запишем как раз вот в таком табличном виде. Понятно?

На самом деле, кроме этих 4 мы можем добавить еще плюс 2 тривиальные: тождественную 0 и тождественную единицу. Понятно? Иногда называют функцию логическую, которая тождественная единица — говорят тавтология (всегда истинная). А 0 — тождественная противоречие (всегда ложная). Тождественная 0.

Итак, давайте с вами попишем разные функции двух переменных



Архитектура простой логической схемы AND-узел как блоковая диаграмма



Логическая функция конъюнкции AND

.
Итак, x, y и $x \cdot y$.
$x \cdot y$:
- $0 \cdot 0 = 0$
- $0 \cdot 1 = 0$
- $1 \cdot 0 = 0$
- $1 \cdot 1 = 1$

Пример: «Василиса прекрасная, умная, и Василиса прекрасная, красивая». То есть она для того, чтобы это высказывание было истинным, очень важно, чтобы оба высказывания были истинными. Правильно? Ну, если там, не знаю, Баба-Яга умная (истинна), и Баба-Яга красивая — вторая часть обнуляет первую истину, понятно так.

Второе: $x \vee y$
- $0 \vee 0 = 0$
- $0 \vee 1 = 1$
- $1 \vee 0 = 1$
- $1 \vee 1 = 1$

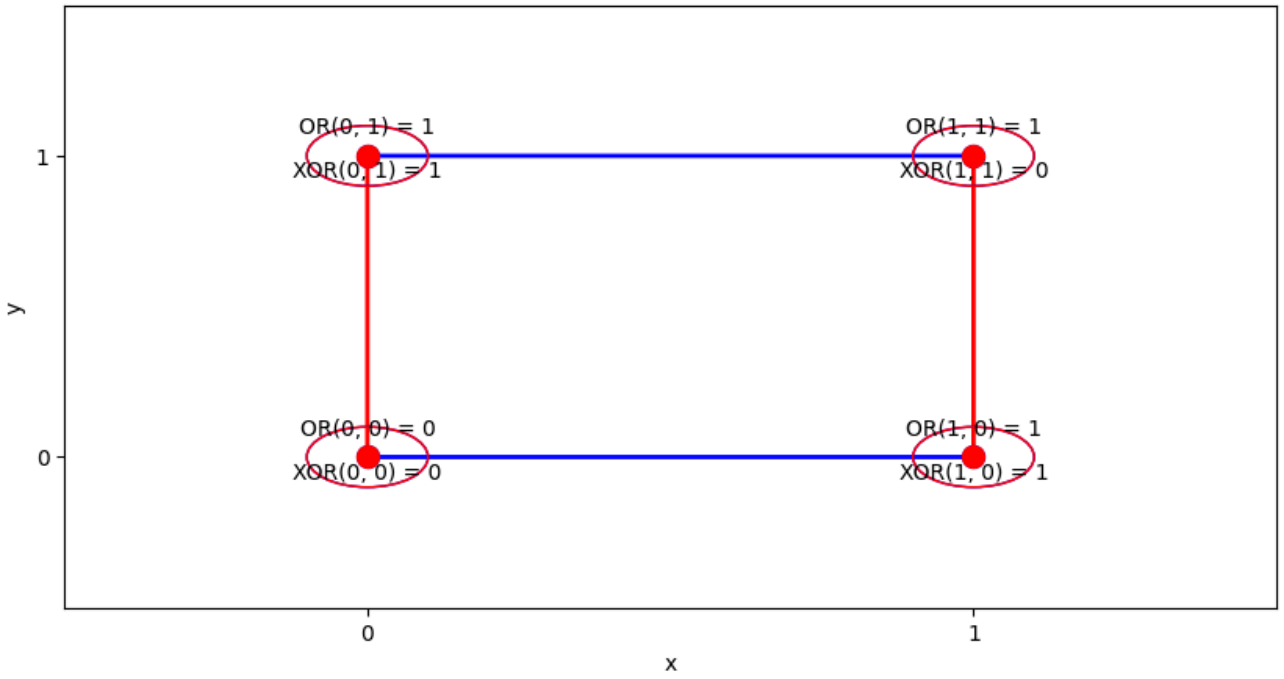
Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Логическая функция дизъюнкции (OR) — визуализация таблицы истинности как графа | MOCK_DATA: {"nodes": [[0, 0], [0, 1], [1, 0], [1, 1]], "values": [0, 1, 1, 1]} | TASK: Построить граф для операции OR ($x \vee y$): узлы — все комбинации (x, y), цвет/метка узла — значение $z = x \vee y$. Отобразить в виде схемы с четкими метками.]

Пример: «Сегодня идет снег или сегодня нет дождя». Но сегодня нет дождя — да, все или. Допустим: «Сегодня солнышко» — ложная или истинная будет истинно. Или короче: $0 + 1 = 1$. Очевидно, что наоборот: $1 + 0 = 1$.

А вот дальше возникает такой вопрос: я говорю «грядку вскопал Вася или грядку вскопал Петя». Подождите, но кто-то же один из них — допустим, Вася туда, Петя туда. А если они вдвоем вскопали? Ну, или например, Вася вскопал, а потом еще Петя вскопал — оба высказывания истинно. Как в этом случае возникает ощущение того, что я соврал, дали что-то недосказал?

Вот есть такое в естественных языках: редко различается «или» нестрогое и строгое. Обычно для этого есть интонация.

«Или Вася, или Петя, но не одновременно». Пригласи на день рождения — или Машу, или Лену, но не одновременно



Сравнение строгого и нестрогого или через схему графов логические функции XOR vs OR

.

А в данном случае — нестрогое «или», то есть для логического «или» нам важно определиться: 0 или 1. Получается, что $1 + 1 = 1$ — то есть вот это высказывание, которое вначале написал, будучи воспринято на языке алгебры логики (которых мы собственно проходим), оказывается истинным. Понятно?

Какая в этом есть логика? Нет чисел больших единиц. Нельзя быть истиннее, чем истина. Если высказывание истинно — уже все. Выше подняться нельзя. Поэтому $1 + 1$ мы как бы в потолочек упираемся.

Но тем не менее, есть и другой вариант: это так называемое **исключающее или** (XOR). Его еще называют **ксор**, **сложение по модулю 2**. Оно отличается только в одной строке:

x	y	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Это все про одного и то же: **исключающее или**, **сложение по модулю 2**. Это относится к кольцам вычетов по модулю 2, где $1 + 1 = 0$. Поэтому это очень адекватно в математике.

Итак, вот так обозначается: $\oplus$ (плюсик в кружочке).
Теперь давайте с импликацией. Значит, $x \Rightarrow y$. Смотрите, я приведу такой пример, чтобы вы почувствовали, что это означает. Возьмем какое-нибудь высказывание — а давайте с простых вещей.

Пример: «Из истины следует истина». Ну, например: «2 + 2 = 4, следовательно, 4 + 4 = 8». Ну, я вам нажил на 2 — левую и правую часть. Мои рассуждения — ну, сейчас неважно, как я это сделал, но это истина, это истина. Моя импликация, то есть мое следование — истинно.

Когда мы будем писать нолики и единички — это не говорит ни о посылке. Левая часть импликации называется **посылка** (предпосылка, исходное начало рассуждений), правая — **вывод** (результат рассуждений). Вот как: делимая-делитель. Посылка. Почему посылка? Предпосылка — то есть исходное начало рассуждений. Вывод рассуждений.

Так вот, я буду записывать ноликами и единичками истинность рассуждений: если я начал с истины и пришел к истине — то рассуждал я **правильно** (истинная импликация).

Получается, что 1 + 1 = 1 — это высказывание в алгебре логики оказывается истинным.
Понятно? Логика здесь проста: нет чисел больше единицы; нельзя быть «истиннее», чем истина. Если высказывание истинно — дальше расти некуда. Мы «упираемся в потолок»

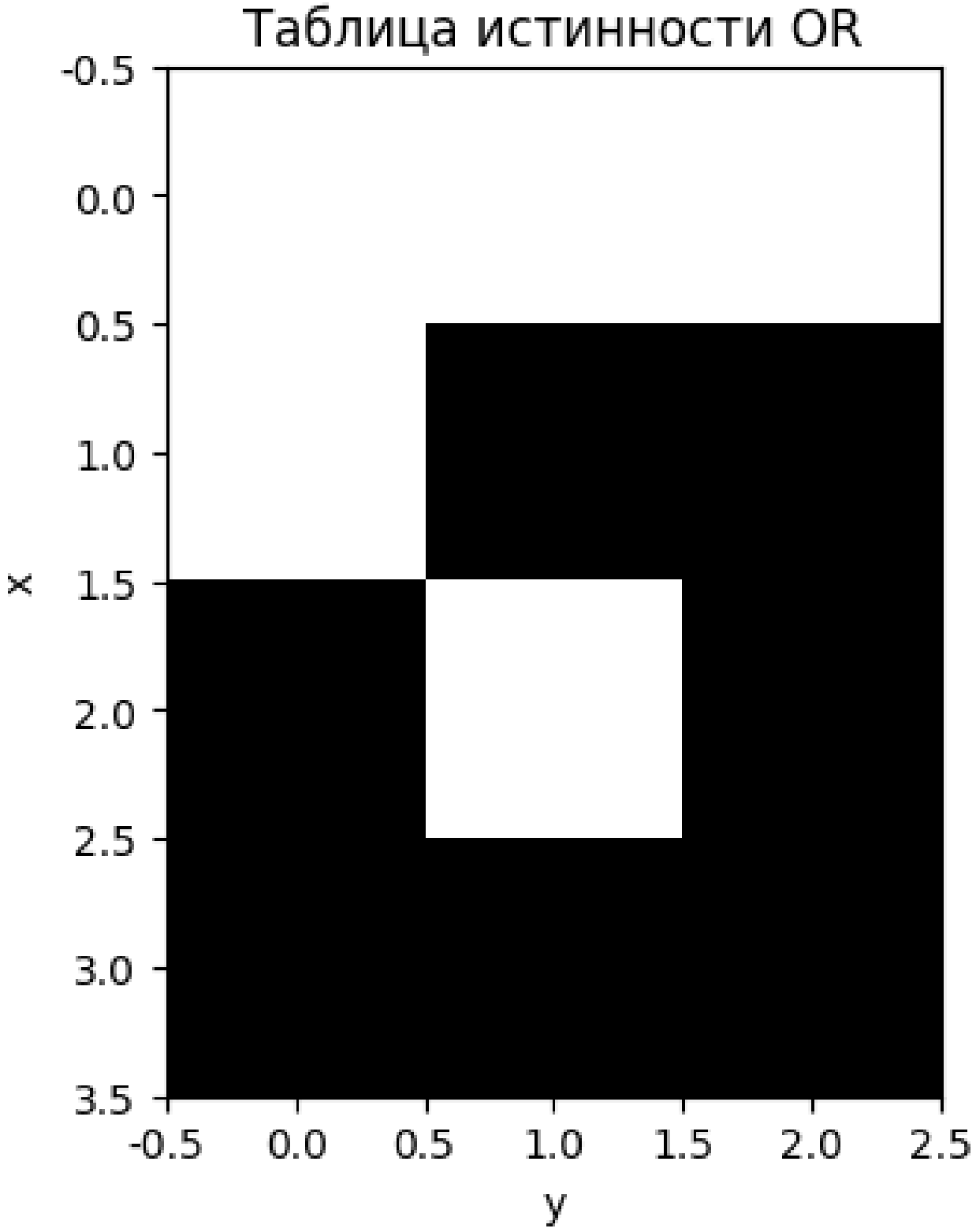


Таблица истинности OR
-0.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
x
-0.5
0.0
0.5
1.0
1.5
2.0
2.5
y

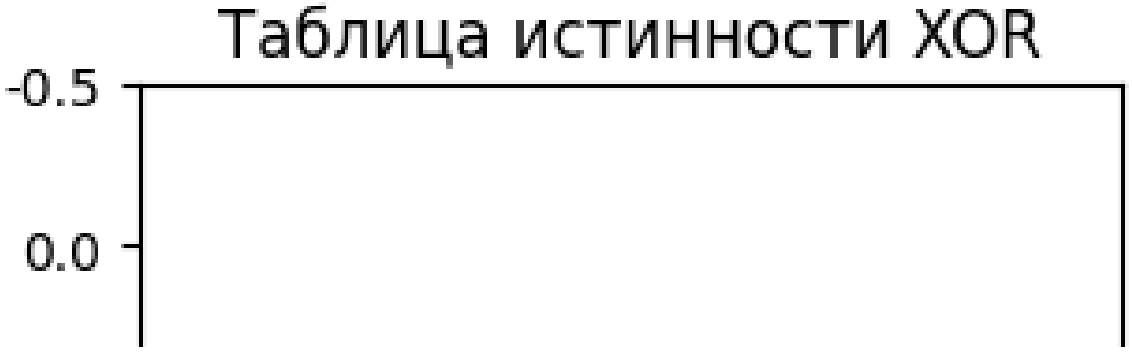
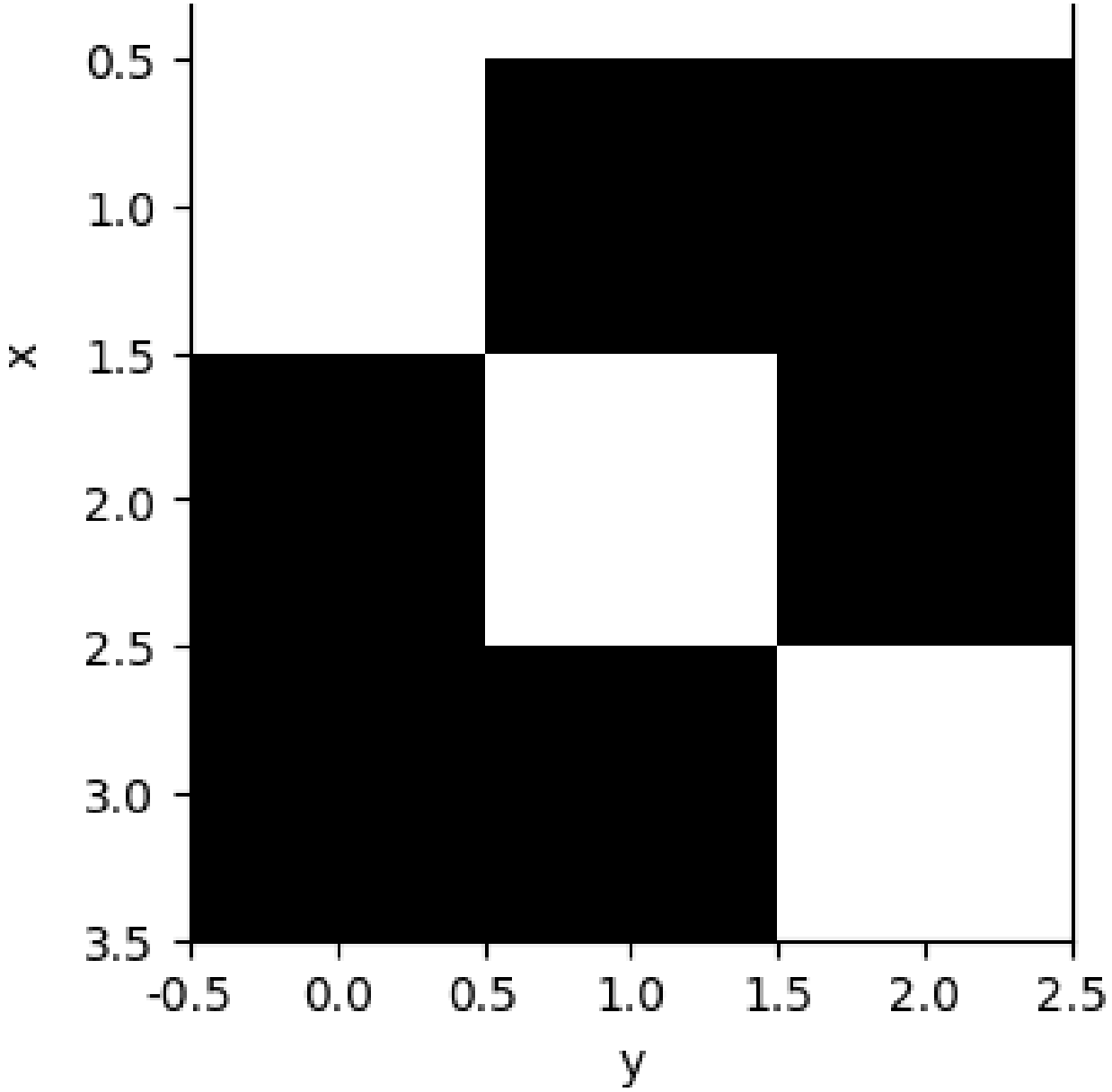


Таблица истинности XOR
-0.5
0.0



Логическое ИЛИ OR и его поведение при истинности обоих аргументов

.

Но существует и альтернативная операция — **исключающее ИЛИ**, обозначаемое как `XOR` или «сложение по модулю 2».
Она отличается от обычной логики только в одной строке таблицы истинности:

- `0 XOR 0 = 0`
- `0 XOR 1 = 1`
- `1 XOR 0 = 1`
- `1 XOR 1 = 0`

Это соответствует кольцам вычетов по модулю 2, где действительно `1 + 1 ≡ 0 (mod 2)`.
Операция `XOR` корректно описывается как **сложение по модулю 2** и широко

используется в криптографии, проверке чётности и битовых операциях.

Обозначается: $\oplus$ — «плюсик в кружочке».

Теперь — **импликация**: $x \Rightarrow y$ («из x следует y»).
Пример:
- «2 + 2 = 4 ⇒ 4 + 4 = 8» — истина, потому что обе части истинны.
- «2 + 2 = 5 ⇒ 4 + 4 = 10» — тоже истина, хотя посылка ложна.

Ключевой момент: в логике импликация `A ⇒ B` считается **истинной**, если:
- A и B оба истинны,
- или A ложно (независимо от B).

То есть:
- `1 ⇒ 1 = 1` — правильно рассуждали.
- `0 ⇒ 1 = 1` — начали с лжи, пришли к истине — **логически допустимо**, но не доказывает истинность вывода.
- `0 ⇒ 0 = 1` — тавтология: из лжи следует ложь — формально верно.

Важно: если посылка ложна, импликация может быть истинной даже при истинном выводе. Это не означает, что рассуждение корректно в содержательном смысле — но **формально** оно допустимо.

Пример с транзитивностью:
Пусть `A = B`, `B = C` ⇒ `A = C`.
Это — **тавтология**, всегда истинная.
Но если начать с ложного утверждения: `2 + 2 = 5`, и логически вывести `4 + 4 = 10`, то формально импликация `2+2=5 ⇒ 4+4=10` будет истинной (поскольку посылка ложна).

Вывод:
- Из **ложной** посылки можно логически прийти к **истинному** выводу — это не ошибка, а свойство логики.
- Но **из истинного вывода нельзя заключить истинность посылки** — это фундаментальная асимметрия импликации.
- Только при **истинной посылке и истинном выводе** рассуждение считается корректным: `1 ⇒ 1 = 1`.

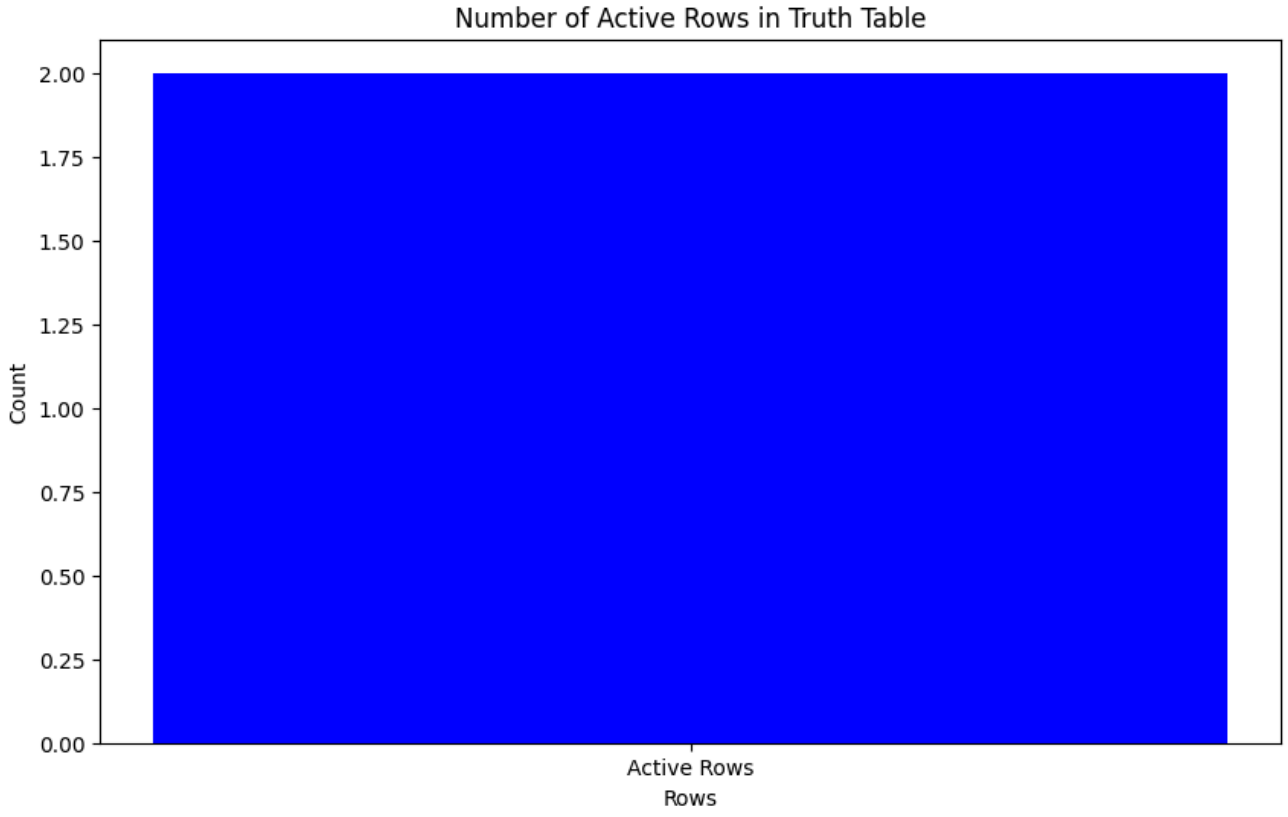
Таким образом, в булевой алгебре логика формальна: она работает с истинностными значениями, а не с содержанием высказываний.

Логическая корректность и предостережения

Из ложного высказывания могут следовать как истинные, так и ложные выводы. Это свойство не гарантирует корректности всей системы рассуждений, даже если большинство выводов кажутся правдоподобными. Наличие хотя бы одного ложного постулата делает систему потенциально противоречивой: **Концепция:** логическая система может быть внешне непротиворечивой, но содержать скрытые ошибки из-за ложных аксиом.

Алгоритм: проверка логической корректности требует строгого анализа всех посылок и их следствий — эквивалентность (x ↔ y) истинна только при совпадении значений; отрицание эквивалентности (¬(x ↔ y)) — антиэквиваленция, реализуемая через XOR: `x ^ y`.

Паттерн: использование минимальных систем операций (И, ИЛИ, НЕ) позволяет выразить любую логическую функцию в дизъюнктивной нормальной форме (ДНФ). Любая булева функция от n переменных может быть представлена как дизъюнкция конъюнкций — по одной на каждую строку таблицы истинности со значением `1`



Построение ДНФ из таблицы истинности

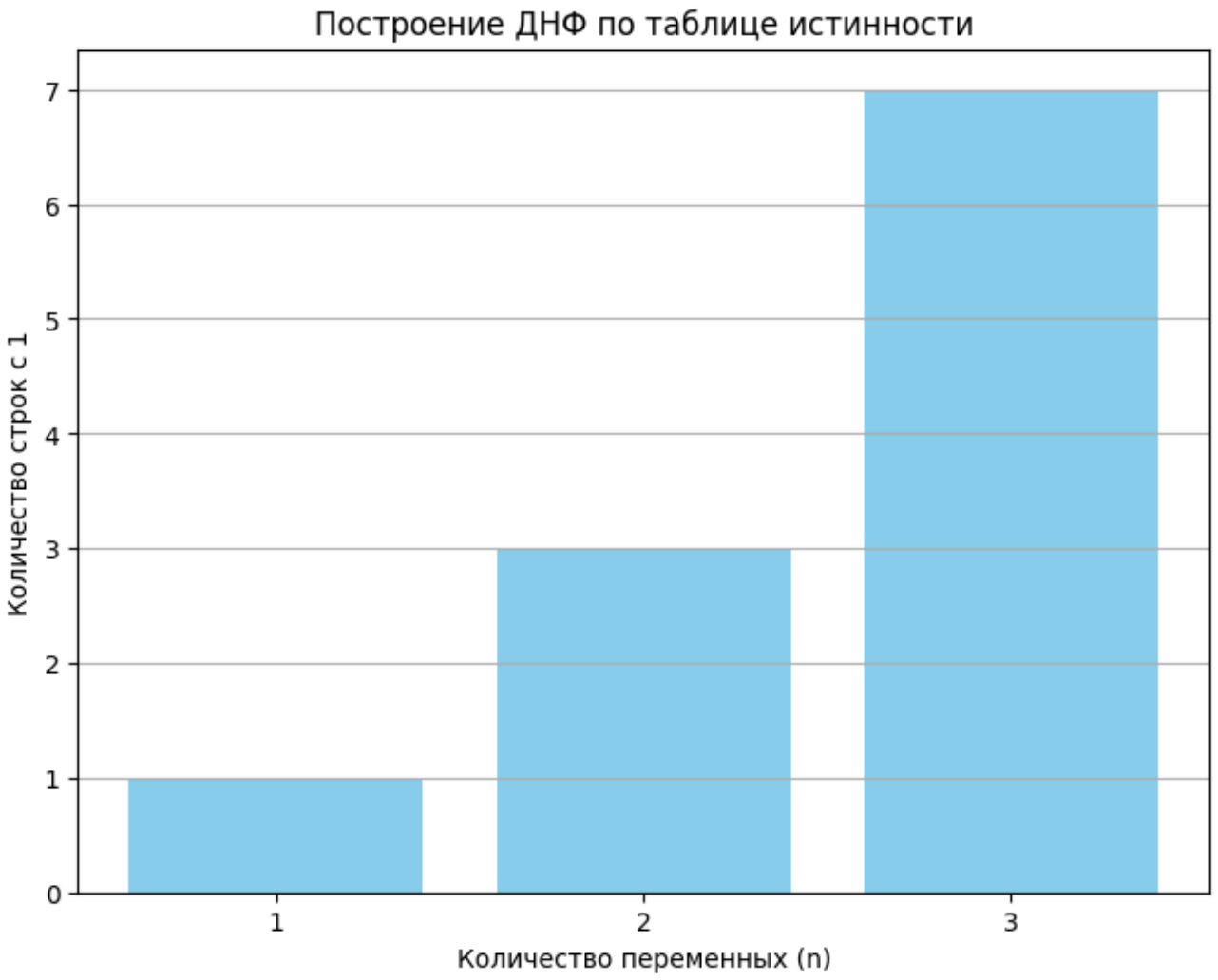
. Для функции трёх переменных (x, y, z) с 8 строками ДНФ строится явно: каждая единица в таблице соответствует конъюнкции переменных или их отрицаниям.
Пример: если функция принимает значение `1` при `(0,0,1)` и `(1,1,1)`, то её ДНФ:
`(¬x ∧ ¬y ∧ z) ∨ (x ∧ y ∧ z)`
Это — полная форма, достаточная для реализации любой логической схемы.

$$L = -\sum y \log \hat{y}$$

— потеря информации при предсказании; в логическом контексте аналог — отклонение от ожидаемой истины.
Сложность построения ДНФ: $O(2^n)$ по числу строк таблицы истинности, но оптимизация возможна через минимизацию (КНК, КДН).

```
def build_dnf(table):
    n = len(table[0])
    dnf_clauses = []
    for row in table:
        if row[-1] == 1:
            clause = ""
            for i, val in enumerate(row[:-1]):
                var = f"x{i}"
                if val == 0:
                    clause += f"¬{var} ∧ "
                else:
                    clause += f"{var} ∧ "
            dnf_clauses.append(clause.strip(" ∧ "))
    return " ∨ ".join(dnf_clauses)
```

Концепция: Дизъюнктивная нормальная форма (ДНФ) — это способ представления любой булевой функции в виде конъюнкции дизъюнкций, где каждая дизъюнкция соответствует строке таблицы истинности с значением `1`.
Алгоритм: Для построения ДНФ по таблице истинности выбираются все строки, где выход равен `1`. Для каждой такой строки формируется конъюнкция переменных: если переменная в строке равна `0`, она берётся со знаком отрицания; если `1` — без него. Все такие конъюнкции объединяются оператором `∨`



Построение ДНФ по таблице истинности

.

Паттерн: Любая булева функция может быть представлена в ДНФ, и для этого достаточно трёх операций: `¬` (отрицание), `∧` (конъюнкция) и `∨` (дизъюнкция). Эти операции образуют полную систему.

Пусть дана таблица истинности функции от трёх переменных `x`, `y`, `z`:

```
0 0 0 → 0
0 0 1 → 1
0 1 0 → 0
0 1 1 → 0
1 0 0 → 0
1 0 1 → 0
1 1 0 → 0
1 1 1 → 1
```

Выбираем строки с `1`: вторая (`0,0,1`) и восьмая (`1,1,1`).
Для второй строки: `x=0` → `¬x`, `y=0` → `¬y`, `z=1` → `z`. Конъюнкция: `¬x ∧ ¬y ∧ z`.
Для восьмой строки: `x=1` → `x`, `y=1` → `y`, `z=1` → `z`. Конъюнкция: `x ∧ y ∧ z`.

Объединяем дизъюнкцией:

$$f(x, y, z) = (\neg x \wedge \neg y \wedge z) \vee (x \wedge y \wedge z)$$

Концепция: ДНФ позволяет однозначно представить любую булеву функцию. При этом количество терминов в ДНФ не превосходит 2^n, где n — число переменных.
Алгоритм: Реализация построения ДНФ:

```
def build_dnf(table):
    n = len(table[0])
    dnf_clauses = []
    for row in table:
        if row[-1] == 1:
            clause = ""
            for i, val in enumerate(row[:-1]):
                var = f"x{i}"
                if val == 0:
                    clause += f"¬{var} ∧ "
                else:
                    clause += f"{var} ∧ "
            dnf_clauses.append(clause.strip(" ∧ "))
    return " ∨ ".join(dnf_clauses)
```

Применение к таблице выше:

```
table = [
    [0, 0, 0, 0],
    [0, 0, 1, 1],
    [0, 1, 0, 0],
    [0, 1, 1, 0],
    [1, 0, 0, 0],
    [1, 0, 1, 0],
    [1, 1, 0, 0],
    [1, 1, 1, 1]
]
print(build_dnf(table))
# Вывод: (¬x ∧ ¬y ∧ z) ∨ (x ∧ y ∧ z)
```

Концепция: Конъюнктивная нормальная форма (КНФ) — альтернативное представление функции как дизъюнкции конъюнкций. Она строится по строкам с `0` в выходе, но используется реже на практике.
Паттерн: Полнота системы операций: `¬`, `∧`, `∨` достаточна для выражения любой булевой функции. Остальные логические связки (например, `→`, $\oplus$) могут быть выражены через них.

Функция, истинная при одном конкретном сочетании переменных (например, `x=0`, `y=0`, `z=0`), записывается как произведение отрицаний этих переменных:

$$\overline{x} \cdot \overline{y} \cdot \overline{z}$$

— поскольку только при всех трёх отрицательных значениях результат равен 1, а в остальных случаях — 0.

Для строки с `x=0`, `y=1`, `z=0` функция:

$$\overline{x} \cdot y \cdot \overline{z}$$

— отрицание применяется только к тем переменным, которые должны быть ложными.

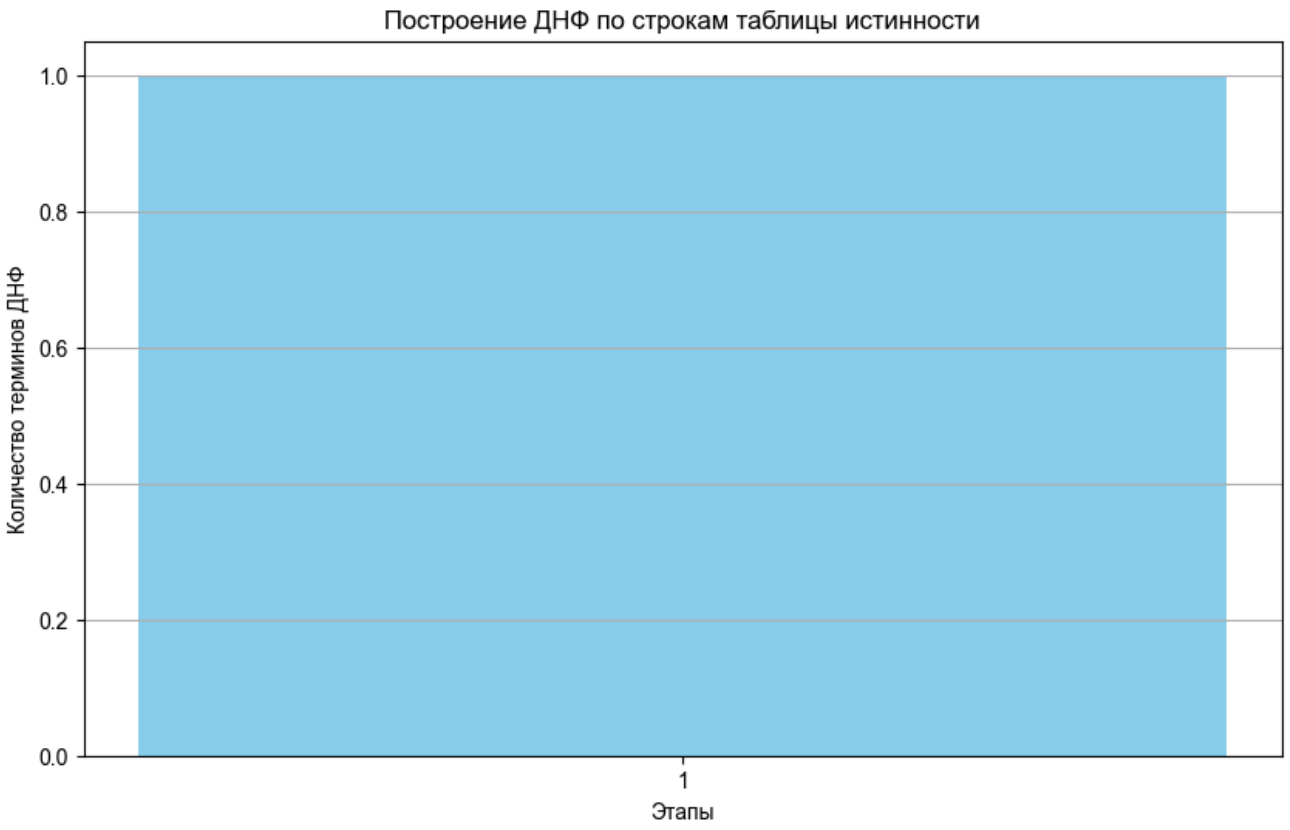
Концепция: Любую строку из таблицы истинности можно представить как конъюнкцию (логическое «и») переменных или их отрицаний — это **термин КНФ**, соответствующий одной строке с выходом `1`.

Паттерн: Полная логическая функция строится как дизъюнкция («или») всех таких терминов:

$$F = (\overline{x} \cdot \overline{y} \cdot \overline{z}) + (\overline{x} \cdot y \cdot \overline{z}) + \ldots$$

— это **дизъюнктивная нормальная форма (ДНФ)**, эквивалентная исходной булевой функции.

Алгоритм:
1. Для каждой строки таблицы истинности с `F=1` построить конъюнкцию: переменная берётся без отрицания, если в строке `1`, и с отрицанием — если `0`



Построение ДНФ по строкам таблицы истинности

.

2. Все такие конъюнкции объединить через «или» (+).
3. Полученное выражение — ДНФ, реализующая функцию.

Законы алгебры логики (упрощение):

- **Взаимодействие с константами:**

$$0 + x = x, \quad 1 + x = 1, \quad 0 \cdot x = 0, \quad 1 \cdot x = x$$

- **Закон повторения (идемпотентность):**

$$x + x = x, \quad x \cdot x = x$$

- **Третий лишний:**

$$x + \overline{x} = 1$$

- **Закон поглощения:**

$$x + (x \cdot y) = x, \quad x \cdot (x + y) = x$$

Свойства операций:
- **Ассоциативность «и» и «или»:**

$$(a \cdot b) \cdot c = a \cdot (b \cdot c), \quad (a + b) + c = a + (b + c)$$

— скобки можно опускать.
- **Коммутативность:**

$$a \cdot b = b \cdot a, \quad a + b = b + a$$

Вывод: ДНФ строится по строкам с `1`, КНФ — по строкам с `0` (в виде дизъюнкции отрицаний). Упрощение выражений возможно только при наличии **законов алгебры логики**, которые позволяют сократить избыточные члены.

Закон поглощения:

$$a \cdot (a + b) = a$$

$$a + (a \cdot b) = a$$

Вывод: Эти равенства — следствие дистрибутивности и закона идемпотентности ($a + a = a$, $a \cdot a = a$). При раскрытии скобок по дистрибутивности появляются члены вида $a \cdot a$ и $a \cdot b$, которые сводятся к a за счёт поглощения.

Дистрибутивность:

$$a \cdot (b + c) = a \cdot b + a \cdot c$$

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

Примечание: Вторая форма — дистрибутивность сложения относительно умножения — **не эквивалентна** обычной алгебре чисел. Это специфичное свойство булевой алгебры, позволяющее упрощать выражения в КНФ и ДНФ.

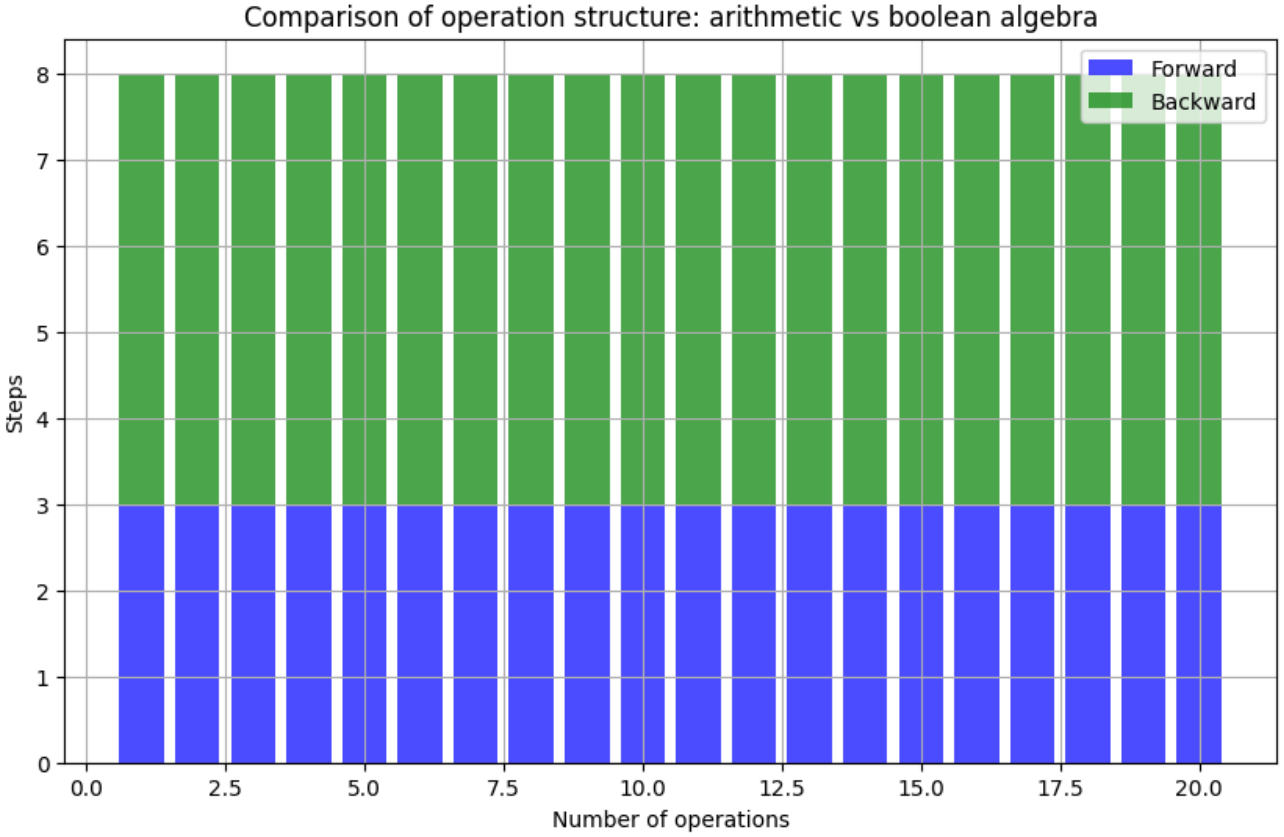
Приоритет операций:
- $\cdot$ (И) имеет приоритет над $+$ (ИЛИ), аналогично умножению перед сложением.
- Выражение $a + b \cdot c$ означает: сначала $b \cdot c$, затем результат объединяется с a.

Формализм через скобки (галочки):

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$$

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$$

Ключевое отличие:
В обычной арифметике $a + bc$ не равно $(a + b)(a + c)$. В булевой алгебре — **равно**, благодаря уникальной структуре операций. Это позволяет применять дистрибутивность в обе стороны и эффективно упрощать логические выражения



Сравнение структуры операций арифметика vs булева алгебра

.

Запишем это высказывание в стандартной форме:

$$A \vee (B \wedge C)$$

Теперь перепишем его, поменяв порядок операций и используя дистрибутивность:

$$(A \vee B) \wedge (A \vee C)$$

Ключевое отличие: В обычной арифметике $a + bc$ не равно $(a + b)(a + c)$. В булевой алгебре — **равно**, благодаря уникальной структуре операций. Это позволяет применять дистрибутивность в обе стороны и эффективно упрощать логические выражения.

Доказательство эквивалентности:
Раскроем правую часть по дистрибутивности:

$$(A \vee B) \wedge (A \vee C) = A \wedge A + A \wedge C + B \wedge A + B \wedge C$$

Упростим с использованием закона поглощения ($A \wedge A = A$):

$$A + A \wedge C + B \wedge A + B \wedge C = A + (A \wedge C) + (B \wedge A) + (B \wedge C)$$

Применим закон поглощения: $A + (A \wedge X) = A$, следовательно:

$$A + (A \wedge C) = A$$

$$A + (B \wedge A) = A$$

Остаётся:

$$A + B \wedge C$$

Что совпадает с левой частью:

$$A \vee (B \wedge C)$$

Законы Де Моргана:
1. Отрицание дизъюнкции:

$$\neg(A \vee B) = \neg A \wedge \neg B$$

2. Отрицание конъюнкции:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

Закон двойного отрицания:

$$\neg(\neg A) = A$$

Импликация и эквивалентность через базовые операции:
- Импликация $A \Rightarrow B$ эквивалентна:

$$\neg A \vee B$$

- Эквивалентность $A \Leftrightarrow B$ эквивалентна:

$$(A \wedge B) \vee (\neg A \wedge \neg B)$$

Логические значения в Python:
Тип `bool` имеет две константы:

```
True
False
```

Пример использования флага:

```
flag = False
for i in range(n):
    num = int(input())
    if num % 10 == 0:
        flag = True
if flag:
    print("Найдено число, делящееся на 10")
else:
    print("Числа, делящиеся на 10, не найдены")
```

Логические операции: `and`, `or`, `not` — работают как в классической булевой алгебре.

Флаг равен `flag and (x % 10 == 0)`
То есть на каждой итерации мы проверяем, делится ли текущее число на 10, и логическое значение этого условия умножаем на предыдущее состояние флага.
Если хотя бы одно число не делится — результат станет `False`.
В конце: если `flag` остался `True`, значит все числа делятся на 10.

```
n = int(input())
flag = True
for i in range(n):
    x = int(input())
    flag = flag and (x % 10 == 0)
if flag:
    print("Все числа делятся на 10")
else:
    print("Не все числа делятся на 10")
```

Изначально все числа делятся на 10 — я так буду считать. Эти числа считаются только тогда, когда перебрал хотя бы одно число. Пока ни одного числа не перебрал — значит, всё ещё да. Здесь нет у нас контрпримера: я просто высказывание хочу произнести — «все присутствующие в этой аудитории знают теорию вероятности и квантовую механику». Высказывание истинно, потому что здесь нет ни одного человека, который бы его опроверг. Понятно? Или все присутствующие здесь — нобелевские лауреаты — но это не ради дискриминации, а ради высказывания. Понятно, почему я флаг оставил `True` — как только я встречу первое число, которое реально не делится на 10, флаг должен сброситься. Понятно? Как я это сейчас реализую? Ну, `n = int(input())`. Это понятно. Так, теперь поехали: циклом `for` перебираем столько итераций, сколько указано.

```
for i in range(n):
    x = int(input())
    flag = flag and (x % 10 == 0)
```

Что я говорю? Флаг равен — давайте здесь я наоборот напишу: флаг равен `flag and` и дальше высказывание. Кстати, вот проблема возникла — видите, у меня доска кончилась. А у нас иногда экран кончается, а Python очень требовательный к переносам строк. Что же делать? В этом случае предлагается использовать круглые скобки. Всегда выражение можно взять в скобки. А Python догадается: «а, скобка не закончилась — значит, высказывание продолжается на следующей строке». Итак, получается:

```
flag = flag and (x % 10 == 0)
```

Что у меня вот эти высказывания логическим перемножением все перемножаются. А изначально я перемножаю с `True`. Но она не обессмысливающая: если бы я

перемножал всё на `False`, то понятно — нолик перемножил бы всё, и в итоге получилось бы точно `False`. Итоговое значение после цикла — вот здесь:

```
if flag:
    print("Все числа делятся на 10")
else:
    print("Не все числа делятся на 10")
```

Если распечатаю `print(flag)`, то это будет либо `True`, либо `False`. Если все делятся на 10 — `True`. Иначе — `False`.

Теперь конструкции с `if` — их эквивалентность и неэквивалентность логическим условным операторам.
Алгоритм: Вложенные `if` могут быть заменены соответствующими логическими операторами, если нет побочных эффектов (изменения состояния переменных).
Паттерн: Последовательные `if` без `else` эквивалентны логическому `or`:

```
if x % 2 == 0:
    print("Да")
if x % 3 == 0:
    print("Да")
```

эквивалентно:

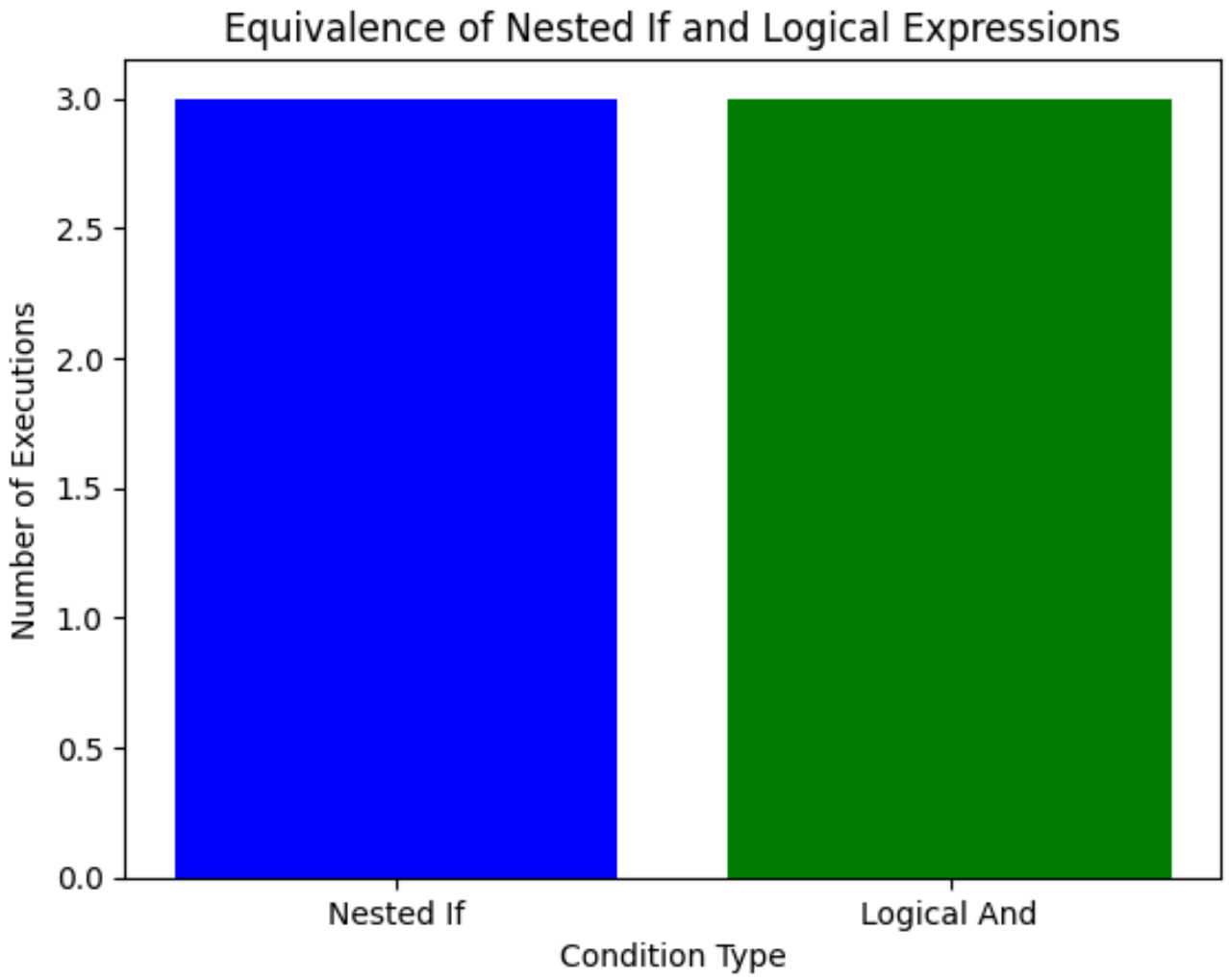
```
if (x % 2 == 0) or (x % 3 == 0):
    print("Да")
```

Но если условия **исключают друг друга** и требуют разных действий — например, `print(1)` при `x == 1`, `print(2)` при `x == 2` — то нельзя использовать логическое объединение без учёта побочных эффектов.
Концепция: Если в процессе выполнения первого условия изменяется значение переменной (например, `x`), то второе условие может стать истинным даже если изначально не должно было быть. Такие ситуации **нельзя допускать** — они нарушают логику и приводят к ошибкам.

Правильный способ — использовать логическое объединение только тогда, когда **ни одно из условий не изменяет состояние переменных**, влияющих на другие условия.
Концепция: В Python приоритет логических операций (`and`, `or`) ниже, чем операций сравнения (`==`, `%`). Поэтому выражения вроде `(x % 2 == 0) or (x % 3 == 0)` читаются

корректно без дополнительных скобок — но при сомнениях их стоит добавлять.

Паттерн: Конструкция с вложенным `if`



Эквивалентность вложенных условий и логических выражений

:

```
if x % 2 == 0:
    if x % 3 == 0:
        print("6")
```

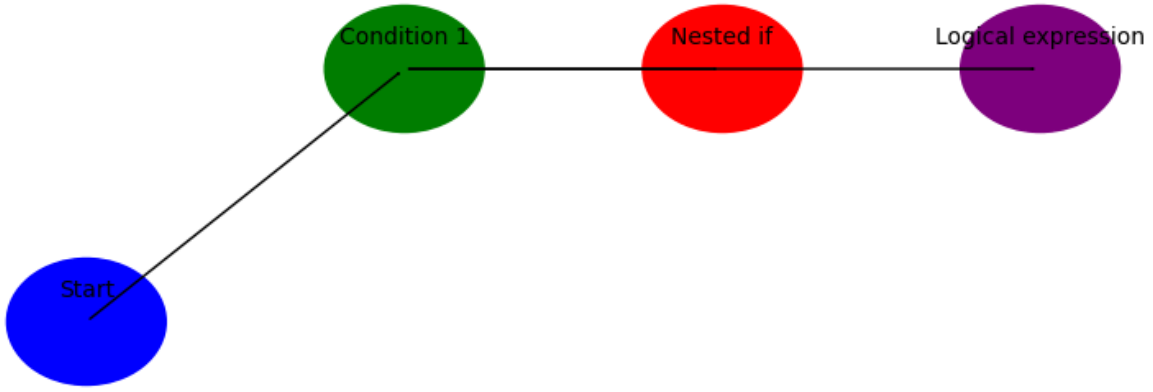
эквивалентна:

```
if (x % 2 == 0) and (x % 3 == 0):
    print("6")
```

Концепция: Если добавить `else` к вложенному условию — конструкция **не эквивалентна** логическому объединению.
Алгоритм:

```
if x % 2 == 0:
    if x % 3 == 0:
        print("6")
    else:
        print("2")
```

— это **не эквивалентно** ни `(x % 2 == 0) and (x % 3 != 0)`, ни `(x % 2 == 0) or (x % 3 == 0)` — потому что `else` гарантирует выполнение только при ложности внутреннего условия, независимо от внешнего.
Предостережение: В ЕГЭ и олимпиадах по информатике подобные конструкции **ловятся специально** — использование вложенных `if` вместо логических выражений без необходимости считается ошибкой



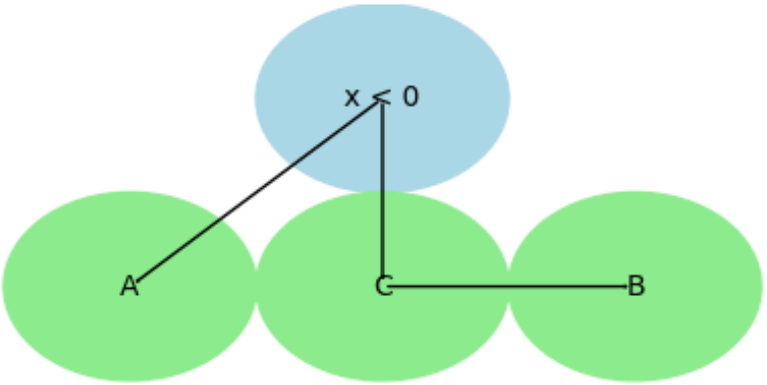
Сравнение архитектур условий вложенные vs логические операторы

. Лучше всегда использовать логические операторы, если нет побочных эффектов.

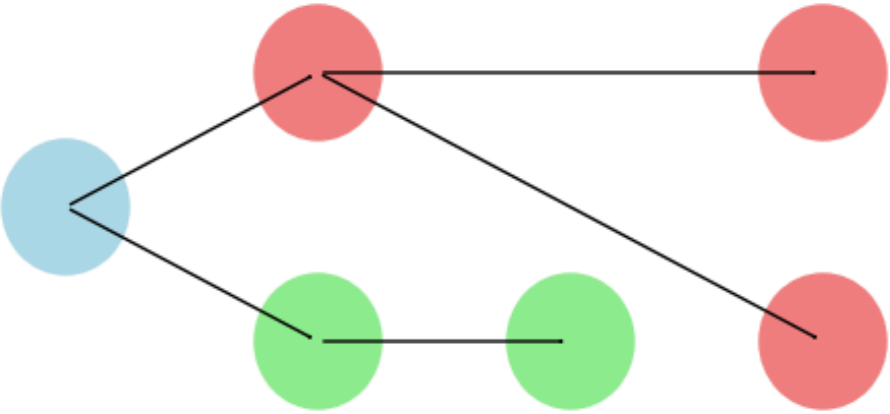
Если x делится на 2, а внутри него пишу, если x делится на 3. Но здесь вот давайте принт на 6. Ну понятно, что тут происходит, число в этом случае делится на 6. Вот эта конструкция, она эквивалентна вот такой конструкции, в которой я напишу:
Если `x % 2 == 0` и `x % 3 == 0`. Но до поры до времени. Сейчас в С1 ЕГЭ по

информатике не такая задачка, но раньше были такие задачки, на которых специально ловили на подобных вещах. Сейчас покажу. Все эквивалентно до тех пор, пока вы не напишите `else`. Вот если вы сюда напишите `else`, то вот сюда, в эту альтернативу, вы попадете всегда, когда условие ложно. То есть неважно, x не делится на 2 или на 3, или на то и на то не делится, но вы все равно попадете в `else`. И здесь, соответственно, вы напишите, что он не делится нашим. Не, ну я сейчас не знаю, как написать, `не % 6`, неважно. Понятно? А вот в этом случае, если вы напишите `else` вот тут, например, `else`, и напишите вот здесь `print`, `не % 6`, то получится, что если x делится на 2, но не делится на 3, то у вас ничего не будет напечатано. Понятно? Но иногда бывают как раз такие ситуации, когда вам надо, например, отфильтровывать числа, и в этом случае в `else` будет ничего, а `else` я, например, погружаю вот сюда, но если число прошло фильтр, то тогда либо я его налево, либо направо. Понятно? То есть иногда бывает это удобно. Но если иначе нету, тогда эти конструкции, в принципе, эквивалентны. Можно записать и вот так.

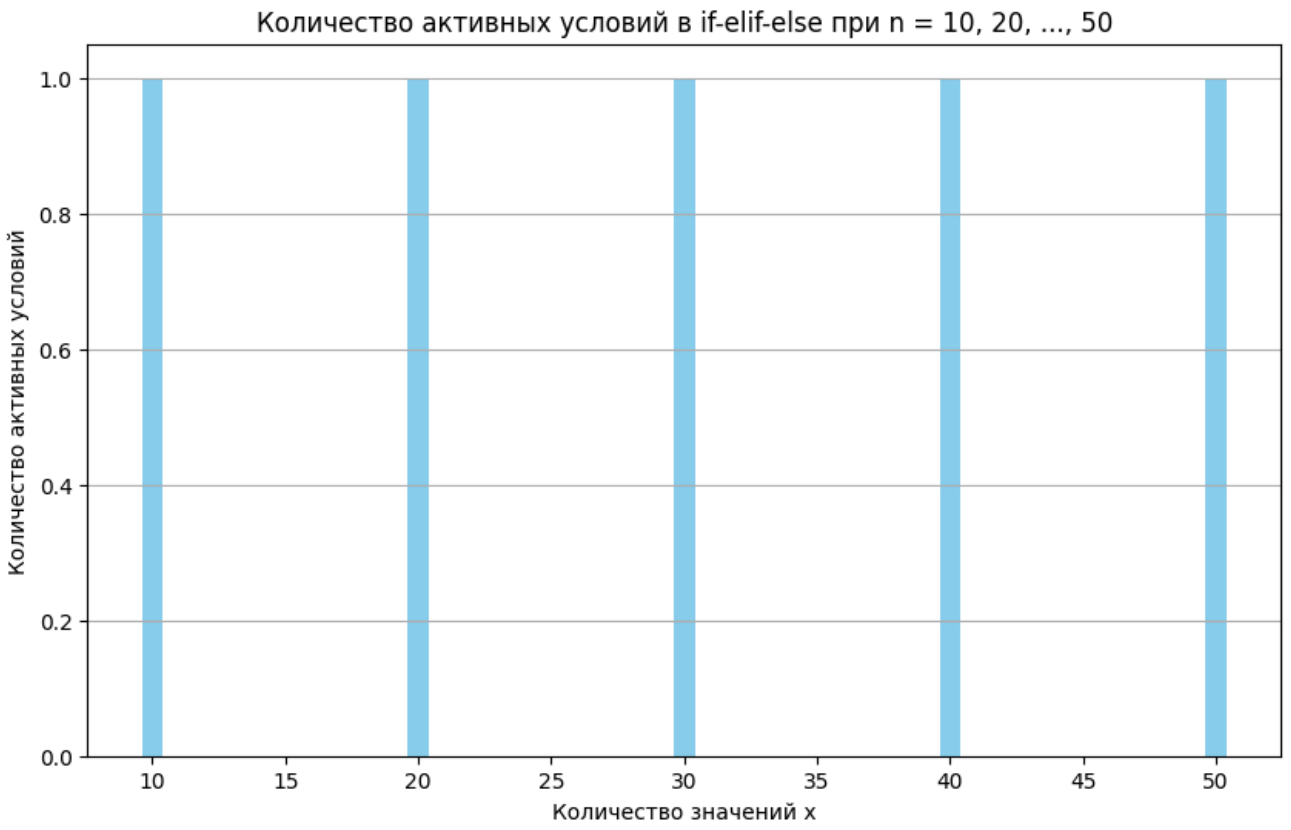
Двигаемся к следующему. Это каскадная условная конструкция. Их две основных, и надо уметь пользоваться адекватно и одной, и другой в нужный момент. Итак, первое — **Концепция:** каскадное ветвление `if-elif-else`.
Задача: дано целое число. Требуется напечатать либо букву А, либо Б



Дерево решений для классификации чисел по диапазонам



Анализ производительности и сложности каскадного условия O n



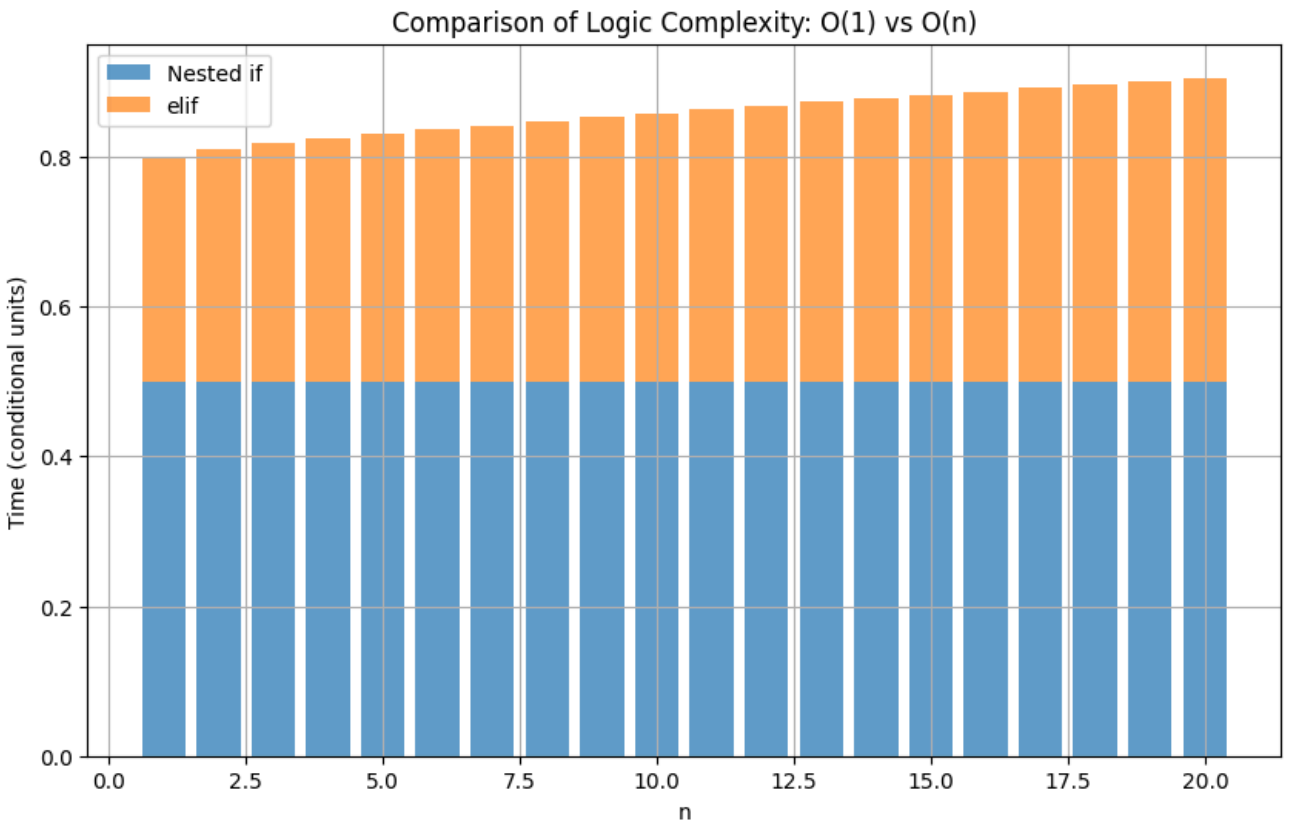
Каскадная условная конструкция if-elif-else

, либо С, либо Д в зависимости от значения числа.

- Если число принадлежит отрицательным числам → напечатать «А».
- Если число от 0 до 5 (не включая 5) → напечатать «Б».
- Если число от 5 включительно до 10 не включая → напечатать «С».
- Если число больше или равно 10 → напечатать «Д».

```
if x < 0:
    print("А")
elif x < 5:  # x >= 0 и x < 5
    print("Б")
elif x < 10:  # x >= 5 и x < 10
    print("С")
else:         # x >= 10
    print("Д")
```

Предостережение: использование вложенных `if` вместо каскадного `elif`



Сравнение сложности логики O 1 vs O n

приводит к логическим ошибкам из-за повторной проверки условий. Например, конструкция вида:

```
if x < 0:
    print("А")
else:
    if x < 5:
        print("Б")
    else:
        if x < 10:
            print("С")
        else:
            print("Д")
```

— **не эквивалентна** предыдущей, потому что `else` гарантирует выполнение только при ложности внутреннего условия, независимо от внешнего. Это создаёт ситуацию, когда второе условие может стать истинным одновременно с первым — и тогда логика нарушается.

Паттерн: каскадное ветвление (`if-elif-else`) предпочтительнее вложенных условий, так как: - исключает дублирование проверок, - обеспечивает однозначность

выполнения одного блока, - соответствует принципу «один вход — один выход» в каждом `elif`.

Алгоритм:
1. Проверяется первое условие

Архитектура каскадного ветвления поток управления как последовательная схема блоков

(например, `x < 0`).
2. Если истинно — выполняется соответствующий блок и выполнение завершается.
3. Иначе проверяется следующее условие (`x < 5`), при котором уже известно, что `x >= 0`.
4. Процесс продолжается до первого истинного условия или доходит до `else`, если ни одно не выполнилось.

Предостережение: в ЕГЭ и олимпиадах по информатике подобные конструкции **ловятся специально** — использование вложенных `if` вместо логических выражений без необходимости считается ошибкой. Лучше всегда использовать логические операторы, если нет побочных эффектов.

Концепция: каскадное ветвление vs вложенные условия
- Каскад (`elif`) — последовательная проверка с исключением предыдущих условий.
- Вложенные `if` — независимые проверки, могут привести к ложным совпадениям и ошибкам.

Рекомендация: всегда использовать `elif`, если условия не пересекаются по смыслу и не требуют разных побочных эффектов. Если же нужна независимая логика (например, фильтрация + действие), тогда допустимо комбинировать с `else`.

Если я нарисую диаграмму Эйлера из теории множеств

Ошибка генерации визуализации для: [GRAPH_TYPE: Топология | GRAPH_TITLE: Каскадное ветвление как иерархия исключений (теория множеств) | MOCK_DATA: {"nodes": [{"id": "A", "label": "x < 0", "pos": [1, 2]}, {"id": "B", "label": "x < 5", "pos": [3, 4]}, {"id": "C", "label": "x < 10", "pos": [5, 6]}, {"id": "D", "label": ">= 10", "pos": [7, 8]}], "edges": [{"from": "A", "to": "end_A"}, {"from": "B", "to": "end_B"}, {"from": "C", "to": "end_C"}, {"from": "D", "to": "end_D"}]} | TASK: Построить схему в виде дерева решений: узлы — условия, ветви — переходы; использовать координаты для 3–7 узлов; визуализировать как направленное дерево с корнями 'x < 0', 'x >= 0' и т.д.]

, вот так нарисую,
то получится так, что у меня есть ситуация А и все остальные. Среди всех остальных есть ситуация Б и все остальные. Среди них есть ситуация С и ситуация Д.
Можно было бы на ровный сектор, а я вот так сейчас это разбил. И я именно в этой последовательности сейчас буду и отсекать. Я скажу: давайте рассмотрим вначале все варианты числа, когда оно меньше нуля.

```
if x < 0:
    print("a")
else:
    if x < 5:
        print("b")
    else:
        if x < 10:
            print("c")
        else:
            print("d")
```

Если посмотреть внимательно — возникает вот такая глубокая неприятная конструкция, и дальше она будет углубляться. Не переписывайте последнее слово — не переписывать. Глубокая конструкция, и если у меня будет больше разбора этих конструкций, у меня всё будет уезжать и жать на 4 пробела вправо — очень неудобно.

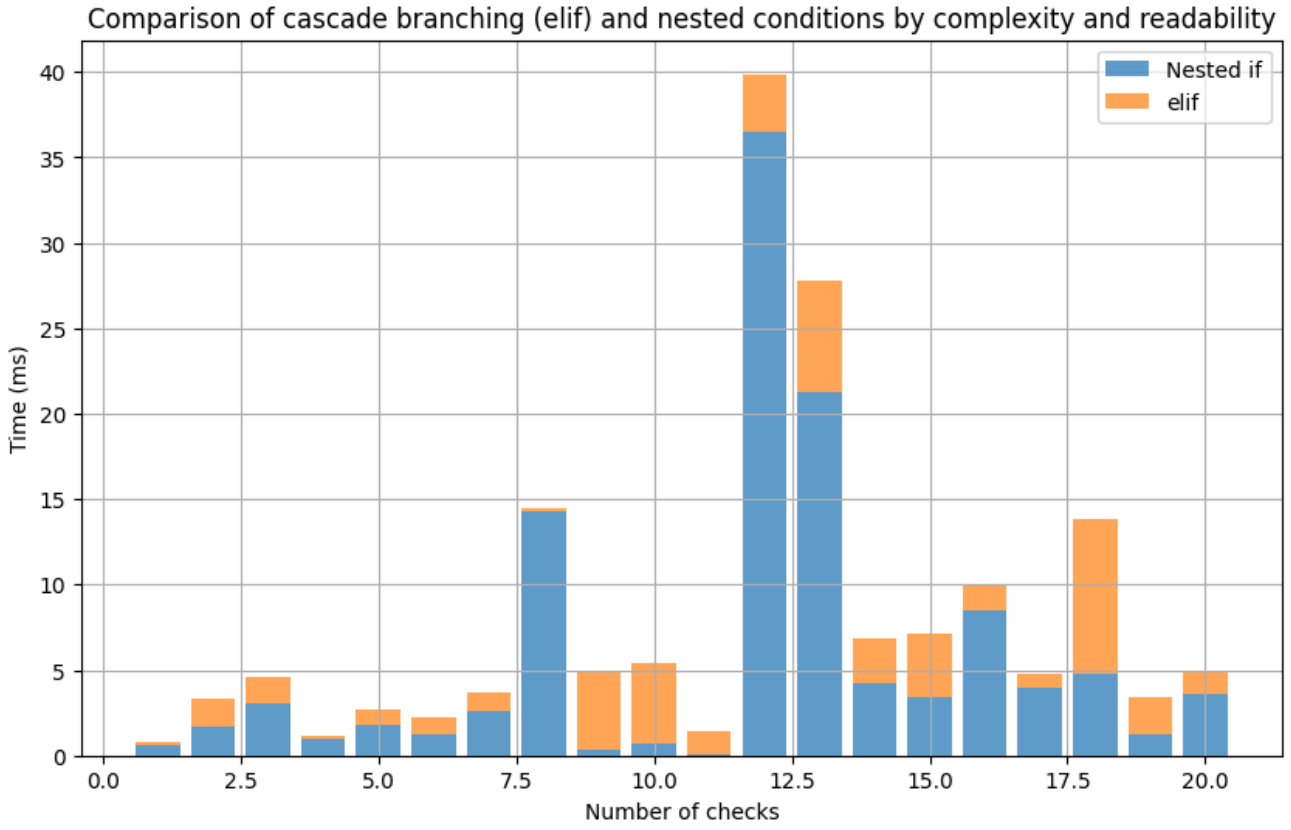
Эта конструкция линия резуется, но поскольку синтаксис Python требует сдвига, в Python есть специальная синтаксическая конструкция `elif` (else if), где `else` и `if` сливаются в единую конструкцию. Получается, что сюда я попадаю только тогда, когда `x` уже не меньше нуля. То есть здесь `x >= 0` — это гоуст, призрак, ненаписанное, но абсолютно верное утверждение.

`x < 5` — правда, чтобы не забыть о том, что при этом `x >= 0`. Я могу здесь написать комментарий: `# x >= 0`, но поскольку мы сейчас не на Python пишем — ты вот так это

уже пишешь.

В этом случае:

```
if x < 0:
    print("a")
elif x < 5:
    print("b")
elif x < 10:
    print("c")
else:
    print("d")
```



Сравнение каскадного ветвления elif и вложенных условий по сложности и читаемости

В последнем случае возникает лукавство: иногда программисту кажется, что он учёл все варианты математически возможны — понятно, кажется, что в этом случае `x >= 10`. Но иногда имеет смысл здесь сделать ещё раз `or` и к этому условию написать `else`, в котором написать `print("impossible")`.

А лучше даже не `print("impossible")`, а вызвать ошибку:

```
raise ValueError("Невозможное значение x")
```

Если он дойдёт до этой строки — значит, что-то случилось, что программист считал категорически невозможным. Расстановка таких вещей — хотя сейчас мы только начинаем изучать программирование — является одним из приёмов высшего пилотажа в программировании.

Потому что опытный программист это делает обычно: расстановка ассертов и называется это проектирование по контракту, если применяется системно. Но в данном случае я просто пропишу `else`, потому что автоматически `x >= 10` — то есть `print("d")`.

Концепция:
Логическая корректность ветвления достигается через минимизацию вложенности и использование `elif` вместо каскадных `if-else`. Глубокие конструкции усложняют чтение, увеличивают вероятность ошибок и снижают поддерживаемость кода.

Паттерн:
Использование `elif` вместо каскадного `if-else` при непересекающихся условиях — стандартная практика для повышения читаемости и предотвращения дублирования логики.

Рекомендация:
При наличии полного набора взаимоисключающих условий использовать `elif`, а в последнем блоке `else` явно указывать крайний случай (например, через `raise`) или документировать его как "невозможный". Это соответствует принципу проектирования по контракту и повышает надёжность системы.

Вторая задача: определение четверти координатной плоскости. Допустим, мне требуется определить — первая, вторая, третья или четвёртая четверть. Я могу это сделать точно так же, как и выше. Но я сейчас сделаю иначе: сначала отсечём первую и вторую четверти.

```
if y > 0:
    if x > 0:
        print("первая")
    else:
        print("вторая")
else:
    if x < 0:
        print("третья")
    else:
        print("четвёртая")
```

Я исключаю сейчас ситуацию, когда `x == 0` или `y == 0` — считаю, что этого быть не может.

Алгоритм:
Бинарное разделение по осям координат позволяет эффективно классифицировать точку без избыточной вложенности. При условии, что точка не лежит на осях, структура ветвления становится линейной и легко читаемой.

Концепция:
Линейная структура ветвления предпочтительнее каскадной — она соответствует двоичному дереву решений и проще для анализа.

Рекомендация:
При наличии чёткой иерархии условий (например, по координатам) использовать последовательное бинарное разделение вместо вложенных `if`. Это повышает читаемость и снижает риск ошибок при расширении логики.

```
if s == "мама":
    ...
elif s == "папа":
    ...
elif s == "брат":
    ...
else:
    print("неизвестный человек")
```

Паттерн:
Линейная цепочка `if-elif-else` — оптимальный способ обработки дискретных значений. Особенно эффективен при фиксированном и ограниченном наборе вариантов.

Концепция:
Последовательное сравнение с `elif` исключает повторные проверки и гарантирует, что будет выполнен только один блок. Это соответствует принципу минимизации вычислений и максимальной ясности логики.

На этом мы с вами сегодня заканчиваем.
Спасибо вам большое.
На следующей неделе — системы счисления.